

Capitolo 04 - Java Enterprise

JDBC Fondamenti

Introduzione

JDBC (Java Database Connectivity) è l'API standard di Java per l'accesso ai database relazionali.

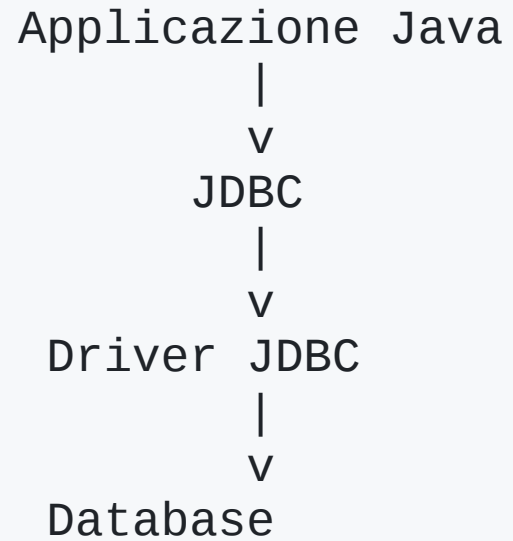
Attraverso JDBC è possibile:

- connettersi a un database
- eseguire query SQL
- inserire, modificare ed eliminare dati
- leggere i risultati delle query
- gestire transazioni

JDBC rappresenta il livello più basso di accesso ai dati in Java e costituisce la base di tecnologie più avanzate come JPA, Hibernate e Spring Data.

Architettura JDBC

L'architettura JDBC prevede quattro componenti principali:



L'applicazione comunica con il database tramite un driver specifico.

Esempi:

Database	Driver
MySQL	mysql-connector-j
PostgreSQL	postgresql
MariaDB	mariadb-java-client
Oracle	ojdbc
SQL Server	mssql-jdbc

Le principali interfacce JDBC

Le classi e interfacce più utilizzate sono:

Classe	Funzione
DriverManager	crea connessioni
Connection	connessione al DB
Statement	esegue SQL
PreparedStatement	SQL parametrizzato
ResultSet	risultati delle query
SQLException	gestione errori

Configurazione del driver

Con Maven:

```
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
  <version>9.0.0</version>  
</dependency>
```

Creare una connessione

URL JDBC

Formato generale:

```
jdbc:tipo_database://host:porta/database
```

Esempio MySQL:

```
jdbc:mysql://localhost:3306/scuola
```

Apertura della connessione

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Main {

    public static void main(String[] args) throws Exception {

        String url = "jdbc:mysql://localhost:3306/scuola";
        String user = "root";
        String password = "password";

        Connection conn =
            DriverManager.getConnection(
                url,
                user,
                password
            );

        System.out.println("Connessione riuscita");

        conn.close();

    }
}
```

Il try-with-resources

Le connessioni devono essere sempre chiuse.

Forma consigliata:

```
try (Connection conn =  
    DriverManager.getConnection(url,user,password)) {  
    System.out.println("Connesso");  
}  
catch (SQLException e) {  
    e.printStackTrace();  
}
```

Alla fine del blocco Java chiuderà automaticamente la connessione.

Eseguire query SELECT

Utilizzo di Statement

```
Statement stmt = conn.createStatement();  
  
ResultSet rs = stmt.executeQuery(  
    "SELECT * FROM studenti");
```

Lettura del ResultSet

```
while(rs.next()) {  
    int id = rs.getInt("id");  
    String nome = rs.getString("nome");  
    System.out.println(id + " " + nome);  
}
```

Output:

```
1 Mario  
2 Anna  
3 Luca
```

Esempio completo

```
try(Connection conn =
    DriverManager.getConnection(url,user,password))
{

    Statement stmt = conn.createStatement();

    ResultSet rs =
        stmt.executeQuery(
            "SELECT * FROM studenti");

    while(rs.next()) {

        System.out.println(
            rs.getInt("id")
            + " "
            + rs.getString("nome"));

    }
}
catch(SQLException e) {
    e.printStackTrace();
}
```

Inserimento dati

INSERT con Statement

```
Statement stmt = conn.createStatement();  
  
stmt.executeUpdate(  
    "INSERT INTO studenti(nome) " +  
    "VALUES('Mario')");
```

Il metodo restituisce il numero di record modificati.

```
int righe = stmt.executeUpdate(sql);
```

Aggiornamento dati

```
String sql =  
    "UPDATE studenti " +  
    "SET nome='Luigi' " +  
    "WHERE id=1";  
  
int righe = stmt.executeUpdate(sql);
```

Eliminazione dati

```
String sql =  
    "DELETE FROM studenti " +  
    "WHERE id=1";  
  
stmt.executeUpdate(sql);
```

Problemi di Statement

Consideriamo:

```
String nome = inputUtente;  
  
String sql =  
    "SELECT * FROM utenti " +  
    "WHERE nome='" + nome + "'";
```

Se l'utente inserisce:

```
' OR 1=1 --
```

la query diventa:

```
SELECT *  
FROM utenti  
WHERE nome=' '  
OR 1=1
```

Tutti i record verranno restituiti.

Questo attacco è chiamato **SQL Injection**.

PreparedStatement

È il meccanismo corretto per eseguire query parametrizzate.

SELECT parametrizzata

```
String sql =  
    "SELECT * FROM studenti " +  
    "WHERE id=?";  
  
PreparedStatement ps =  
    conn.prepareStatement(sql);  
  
ps.setInt(1, 10);  
  
ResultSet rs =  
    ps.executeQuery();
```

INSERT parametrizzata

```
String sql =  
    "INSERT INTO studenti(nome) " +  
    "VALUES(?)";  
  
PreparedStatement ps =  
    conn.prepareStatement(sql);  
  
ps.setString(1, "Mario");  
  
ps.executeUpdate();
```


UPDATE parametrizzato

```
String sql =  
    "UPDATE studenti " +  
    "SET nome=? " +  
    "WHERE id=?";  
  
PreparedStatement ps =  
    conn.prepareStatement(sql);  
  
ps.setString(1, "Luigi");  
ps.setInt(2, 5);  
  
ps.executeUpdate();
```

DELETE parametrizzato

```
String sql =  
    "DELETE FROM studenti " +  
    "WHERE id=?";  
  
PreparedStatement ps =  
    conn.prepareStatement(sql);  
  
ps.setInt(1, 5);  
  
ps.executeUpdate();
```

Tipi supportati

Metodo	Tipo Java
setInt()	int
setLong()	long
setDouble()	double
setString()	String
setDate()	Date
setBoolean()	boolean

Gestione delle eccezioni

```
try {  
    ...  
}  
catch(SQLException e) {  
    System.out.println(  
        e.getMessage());  
  
    System.out.println(  
        e.getErrorCode());  
  
    System.out.println(  
        e.getSQLState());  
}
```

Best Practice

Utilizzare sempre PreparedStatement

Evita:

```
Statement stmt =  
    conn.createStatement();
```

Preferire:

```
PreparedStatement ps =  
    conn.prepareStatement(sql);
```

Chiudere sempre le risorse

Usare:

```
try-with-resources
```

per:

- Connection
- Statement
- PreparedStatement
- ResultSet

Separare SQL e logica Java

Evitare:

```
String sql =  
"SELECT * FROM studenti " +  
"WHERE nome='" + nome + "'";
```

Preferire:

```
String sql =  
"SELECT * FROM studenti " +  
"WHERE nome=?";
```


Errori comuni

DimENTICARE rs.next()

```
rs.getString("nome");
```

Corretto:

```
if(rs.next()) {  
    rs.getString("nome");  
}
```

Non chiudere le connessioni

Produce:

- memory leak
- connessioni esaurite
- rallentamenti

Costruire query concatenando stringhe

```
String sql =  
"SELECT * FROM utenti " +  
"WHERE nome='" + nome + "'";
```

Rischio SQL Injection.

Esercizi

Esercizio 1

Connettersi a un database MySQL e stampare:

```
Connessione riuscita
```

Esercizio 2

Leggere tutti i record della tabella:

```
studenti
```

e visualizzare:

```
id nome
```

Esercizio 3

Inserire un nuovo studente utilizzando `PreparedStatement` .

Esercizio 4

Aggiornare il nome di uno studente dato l'id.

Esercizio 5

Eliminare uno studente dato l'id.

Esercizio 6

Realizzare una classe:

```
StudenteDAO
```

contenente i metodi:

```
findAll()  
findById()  
insert()  
update()  
delete()
```

Domande di ripasso

1. Che cos'è JDBC?
2. Qual è il ruolo del driver JDBC?
3. A cosa serve Connection?
4. Qual è la differenza tra Statement e PreparedStatement?
5. Che cos'è un ResultSet?
6. Perché PreparedStatement previene la SQL Injection?
7. Quando utilizzare executeQuery()?
8. Quando utilizzare executeUpdate()?
9. Quali risorse devono essere chiuse?
10. Quali vantaggi offre il try-with-resources?

JDBC Avanzato

Introduzione

Dopo aver imparato a utilizzare JDBC per eseguire operazioni CRUD di base, è importante conoscere alcune funzionalità avanzate che permettono di realizzare applicazioni robuste e professionali:

- transazioni
- commit e rollback
- savepoint
- batch processing
- generated keys
- metadata
- connection pooling
- pattern DAO

Questi argomenti sono fondamentali nello sviluppo enterprise e costituiscono la base delle tecnologie moderne come Hibernate e Spring Data.

Le transazioni

Cos'è una transazione

Una transazione è un insieme di operazioni che devono essere eseguite come un'unica unità logica.

Esempio:

Bonifico bancario

1. Preleva 100€
2. Versa 100€

Entrambe le operazioni devono riuscire.

Se una fallisce, devono essere annullate entrambe.

Proprietà ACID

Le transazioni rispettano le proprietà ACID.

Atomicity

Tutto o niente.

Consistency

Il database rimane consistente.

Isolation

Le transazioni non interferiscono tra loro.

Durability

I dati confermati rimangono persistenti.

Auto Commit

Per default JDBC lavora in modalità:

```
autoCommit = true
```

Ogni istruzione SQL viene salvata immediatamente.

```
stmt.executeUpdate(sql);
```

equivale implicitamente a:

```
executeUpdate();  
commit();
```

Verificare l'impostazione

```
System.out.println(  
    conn.getAutoCommit());
```

Disabilitare Auto Commit

```
conn.setAutoCommit(false);
```

Da questo momento il programmatore controlla esplicitamente la transazione.

Commit

Il commit rende permanenti le modifiche.

```
conn.commit();
```

Esempio

```
conn.setAutoCommit(false);  
  
stmt.executeUpdate(sql1);  
stmt.executeUpdate(sql2);  
  
conn.commit();
```

Le modifiche vengono salvate solo alla fine.

Rollback

Annulla tutte le modifiche effettuate nella transazione.

```
conn.rollback();
```

Esempio completo

```
try {  
    conn.setAutoCommit(false);  
    stmt.executeUpdate(sql1);  
    stmt.executeUpdate(sql2);  
    conn.commit();  
}  
catch(Exception e) {  
    conn.rollback();  
}
```

Caso reale: bonifico

```
conn.setAutoCommit(false);

try {

    String prelievo =
        ""
        UPDATE conti
        SET saldo = saldo - 100
        WHERE id = 1
        "";

    String versamento =
        ""
        UPDATE conti
        SET saldo = saldo + 100
        WHERE id = 2
        "";

    stmt.executeUpdate(prelievo);
    stmt.executeUpdate(versamento);

    conn.commit();

}
catch(Exception e) {

    conn.rollback();

}
```

Savepoint

Permette di creare punti di ripristino intermedi.

```
Savepoint sp =  
    conn.setSavepoint();
```

Rollback parziale

```
conn.rollback(sp);
```

Esempio

```
conn.setAutoCommit(false);  
stmt.executeUpdate(sql1);  
Savepoint sp =  
    conn.setSavepoint();  
stmt.executeUpdate(sql2);  
conn.rollback(sp);  
conn.commit();
```

Il primo aggiornamento rimane.

Il secondo viene annullato.

Batch Processing

Perché usare il batch

Inserire 10.000 record con 10.000 query separate è inefficiente.

Meglio inviarli in gruppo.

Esempio

```
PreparedStatement ps =  
    conn.prepareStatement(  
        ""  
        "INSERT INTO studenti(nome)  
        VALUES(?  
        """);  
  
for(int i=1;i<=1000;i++) {  
    ps.setString(1,"Studente " + i);  
    ps.addBatch();  
}  
---  
ps.executeBatch();
```

Vantaggi

- meno traffico di rete
- maggiore velocità
- migliore scalabilità

Generated Keys

Molte tabelle utilizzano:

```
id INT AUTO_INCREMENT
```

Spesso è necessario recuperare l'id appena creato.

Creazione

```
PreparedStatement ps =  
    conn.prepareStatement(  
        sql,  
        Statement.RETURN_GENERATED_KEYS);
```

Recupero

```
ResultSet rs =  
    ps.getGeneratedKeys();  
  
if(rs.next()) {  
    int id = rs.getInt(1);  
    System.out.println(id);  
}
```

Esempio completo

```
String sql =  
    ""  
    INSERT INTO studenti(nome)  
    VALUES(?)  
    "";  
  
PreparedStatement ps =  
    conn.prepareStatement(  
        sql,  
        Statement.RETURN_GENERATED_KEYS);  
  
ps.setString(1, "Mario");  
  
ps.executeUpdate();  
  
ResultSet rs =  
    ps.getGeneratedKeys();  
  
if(rs.next()) {  
    int id = rs.getInt(1);  
  
    System.out.println("ID = " + id);  
}
```


Database Metadata

JDBC può interrogare il database stesso.

```
DatabaseMetaData meta =  
    conn.getMetaData();
```

Informazioni disponibili

```
System.out.println(  
    meta.getDatabaseProductName());  
  
System.out.println(  
    meta.getDatabaseProductVersion());  
  
System.out.println(  
    meta.getDriverName());
```

Output

```
MySQL  
9.0  
MySQL JDBC Driver
```

ResultSet Metadata

Permette di analizzare dinamicamente una query.

```
ResultSetMetaData meta =  
    rs.getMetaData();
```

Numero colonne

```
int colonne =  
    meta.getColumnCount();
```

Elenco colonne

```
for(int i=1;i<=colonne;i++) {  
    System.out.println(  
        meta.getColumnName(i));  
}
```

Tipi di ResultSet

Forward Only

È il tipo predefinito.

```
rs.next();
```

Si può avanzare soltanto.

Scrollable

Permette di muoversi avanti e indietro.

```
Statement stmt =  
    conn.createStatement(  
        ResultSet.TYPE_SCROLL_INSENSITIVE,  
        ResultSet.CONCUR_READ_ONLY);
```

Esempio

```
rs.last();  
  
System.out.println(  
    rs.getRow());  
  
rs.first();
```

Connection Pooling

Problema

Aprire una connessione è costoso.

```
DriverManager.getConnection(...)
```

richiede:

- autenticazione
- allocazione risorse
- comunicazione col DB

Soluzione

Riutilizzare connessioni esistenti.



Vantaggi

- maggiore velocità
- minore carico sul database
- migliore scalabilità

Librerie comuni

- HikariCP
- Apache DBCP
- C3P0

Oggi HikariCP è lo standard de facto.

DAO Pattern

Problema

Mescolare SQL e logica applicativa crea codice difficile da mantenere.

Esempio da evitare:

```
public class Main {  
    public static void main(String[] args)  
        throws Exception {  
        Connection conn = ...  
        PreparedStatement ps = ...  
        ...  
    }  
}
```

Soluzione

Creare una classe DAO.

DAO significa:

Data Access Object

Entity

```
public class Studente {  
    private int id;  
    private String nome;  
  
}
```

DAO

```
public interface StudenteDAO {  
    List<Studente> findAll();  
    Optional<Studente> findById(int id);  
    void insert(Studente s);  
    void update(Studente s);  
    void delete(int id);  
}
```

Implementazione

```
public class StudenteDAOImpl  
    implements StudenteDAO {  
  
}
```

Benefici

- separazione responsabilità
- codice più leggibile
- maggiore testabilità
- riuso del codice

Best Practice

Utilizzare sempre PreparedStatement

Mai concatenare stringhe SQL.

Chiudere le risorse

Utilizzare sempre:

```
try-with-resources
```

Gestire le transazioni

Quando più query devono essere eseguite insieme:

```
setAutoCommit(false)
```

Centralizzare l'accesso ai dati

Usare DAO o Repository.

Evitare SQL sparso

Preferire:

```
StudenteDAO
```

anziché query distribuite in tutto il progetto.

Errori comuni

Dimenticare il commit

```
conn.setAutoCommit(false);
```

senza:

```
conn.commit();
```

Le modifiche verranno perse.

Dimenticare il rollback

In caso di errore il database può restare in stato inconsistente.

Non usare batch

Per grandi volumi di dati:

```
executeBatch()
```

è molto più efficiente.

Recuperare male le generated keys

Occorre creare il PreparedStatement con:

```
Statement.RETURN_GENERATED_KEYS
```

Esercizi

Esercizio 1

Creare una transazione che esegua due UPDATE e un COMMIT finale.

Esercizio 2

Simulare un bonifico tra due conti utilizzando rollback in caso di errore.

Esercizio 3

Inserire 1000 studenti usando il Batch Processing.

Esercizio 4

Inserire uno studente e recuperare l'ID generato automaticamente.

Esercizio 5

Visualizzare le informazioni del database tramite DatabaseMetaData.

Esercizio 6

Stampare dinamicamente i nomi delle colonne di una query usando `ResultSetMetaData`.

Esercizio 7

Progettare una classe `StudenteDAO` con:

```
findAll()  
findById()  
insert()  
update()  
delete()
```

Domande di ripasso

1. Cos'è una transazione?
2. Cosa significa ACID?
3. Qual è la differenza tra commit e rollback?
4. A cosa serve un Savepoint?
5. Quando conviene usare il Batch Processing?
6. Come si recupera una generated key?
7. Cos'è DatabaseMetaData?
8. Cos'è ResultSetMetaData?
9. Perché il Connection Pooling migliora le prestazioni?
10. Qual è il ruolo del pattern DAO?

Maven Basi

Introduzione

Quando un progetto Java cresce, diventa difficile gestire manualmente:

- librerie esterne
- compilazione
- packaging
- testing
- distribuzione

Per risolvere questi problemi si utilizzano strumenti di **Build Automation**.

Il più diffuso nel mondo Java è **Apache Maven**.

Maven consente di:

- gestire le dipendenze
- compilare il progetto
- eseguire i test

Cos'è Maven

Maven è un sistema di gestione progetti basato sul concetto di:

`Convention over Configuration`

Invece di configurare tutto manualmente, Maven assume una struttura standard.

Vantaggi

- gestione automatica delle librerie
- build riproducibili
- standardizzazione
- integrazione con IDE
- integrazione con CI/CD

Installazione

Verifica installazione:

```
mvn -version
```

Output tipico:

```
Apache Maven 3.x.x
```

```
Java version: 21
```

Creazione di un progetto Maven

```
mvn archetype:generate
```

Oppure tramite IDE:

- IntelliJ IDEA
- Eclipse
- NetBeans
- VS Code

Struttura standard di un progetto Maven

```
progetto
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   └── resources
    └── test
        ├── java
        └── resources
```

src/main/java

Contiene il codice sorgente.

```
src/main/java
```

src/main/resources

Contiene:

- file properties
- configurazioni
- template
- immagini

Esempio:

```
application.properties
```

src/test/java

Contiene i test automatici.

src/test/resources

Risorse utilizzate dai test.

Il file pom.xml

È il cuore del progetto Maven.

POM significa:

Project Object Model

Esempio minimo

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>it.corsojava</groupId>  
  <artifactId>demo</artifactId>  
  <version>1.0.0</version>  
</project>
```

Coordinate Maven

Ogni progetto è identificato da:

groupId

Identifica organizzazione o azienda.

```
<groupId>it.corsojava</groupId>
```

artifactId

Nome del progetto.

```
<artifactId>gestione-studenti</artifactId>
```

version

Versione dell'applicazione.

```
<version>1.0.0</version>
```

Coordinate complete

```
it.corsojava:gestione-studenti:1.0.0
```

Dipendenze

Una dipendenza rappresenta una libreria esterna.

Esempio:

```
<dependencies>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.0.0</version>
  </dependency>
</dependencies>
```

Cosa succede

Alla prima build Maven:

1. scarica la libreria
2. scarica le dipendenze necessarie
3. le salva in cache locale

Repository

Local Repository

Cartella locale:

```
~/.m2/repository
```

Contiene tutte le librerie scaricate.

Central Repository

Repository pubblico Maven.

```
https://repo.maven.apache.org
```

Da qui vengono scaricate la maggior parte delle librerie.

Ciclo di vita (Lifecycle)

Maven organizza il processo di build in fasi.

Le più importanti sono:

```
validate  
compile  
test  
package  
verify  
install  
deploy
```

validate

Verifica la correttezza del progetto.

```
mvn validate
```

compile

Compila il codice.

```
mvn compile
```

Output:

```
target/classes
```

test

Esegue i test.

```
mvn test
```

package

Genera l'artefatto finale.

```
mvn package
```

Produce:

```
target/
```

con:

```
app.jar
```

oppure:

```
app.war
```

install

Installa l'artefatto nel repository locale.

```
mvn install
```

Permette ad altri progetti Maven locali di usarlo.

deploy

Pubblica l'artefatto in un repository remoto.

```
mvn deploy
```

Maven e Java

Specificare la versione Java:

```
<properties>  
    <maven.compiler.source>21</maven.compiler.source>  
    <maven.compiler.target>21</maven.compiler.target>  
</properties>
```

Maven Compiler Plugin

Configurazione moderna:

```
<build>
  <plugins>
    <plugin>
      <groupId>
        org.apache.maven.plugins
      </groupId>
      <artifactId>
        maven-compiler-plugin
      </artifactId>
      ---
      <version>3.13.0</version>
      <configuration>
        <release>21</release>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Maven e JUnit

Aggiungere JUnit:

```
<dependency>  
    <groupId>org.junit.jupiter</groupId>  
    <artifactId>junit-jupiter</artifactId>  
    <version>5.12.0</version>  
    <scope>test</scope>  
</dependency>
```

Esempio completo di pom.xml

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>it.corsojava</groupId>
  <artifactId>gestione-studenti</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    ---
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.0.0</version>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.12.0</version>
      <scope>test</scope>
    ---
    </dependency>
  </dependencies>
</project>
```

Il direttorio target

Maven genera tutto in:

```
target/
```

Contiene:

- classi compilate
- test compilati
- JAR
- WAR
- report

Generalmente va escluso dal versionamento Git.

Maven e Git

Tipico file `.gitignore` :

```
target/  
.idea/  
.project  
.classpath  
.settings/
```


Best Practice

Non salvare librerie nel progetto

Evitare:

```
lib/mysql.jar
```

Usare sempre Maven.

Versionare il pom.xml

Il file più importante del progetto.

Usare una struttura standard

Seguire sempre:

```
src/main/java  
src/test/java
```

Tenere aggiornate le dipendenze

Corregge bug e vulnerabilità.

Errori comuni

Modificare manualmente target

La cartella viene rigenerata.

Salvare JAR nel repository Git

Aumenta inutilmente le dimensioni.

Dimenticare mvn test

I test dovrebbero essere eseguiti regolarmente.

Coordinate duplicate

Due dipendenze incompatibili possono generare conflitti.

Esercizi

Esercizio 1

Installare Maven e verificare:

```
mvn -version
```

Esercizio 2

Creare un nuovo progetto Maven.

Esercizio 3

Aggiungere il driver MySQL al progetto.

Esercizio 4

Aggiungere JUnit come dipendenza.

Esercizio 5

Compilare il progetto:

```
mvn compile
```


Esercizio 6

Generare il JAR:

```
mvn package
```

Esercizio 7

Analizzare la cartella:

```
target/
```

e identificare i file generati.

Domande di ripasso

1. Cos'è Maven?
2. Quali problemi risolve?
3. Cos'è il file `pom.xml` ?
4. Cosa rappresentano `groupId`, `artifactId` e `version` ?
5. Dove vengono scaricate le dipendenze?
6. Qual è la differenza tra `compile` e `package` ?
7. A cosa serve `install` ?
8. Dove vengono generati gli artefatti?
9. Perché si usa JUnit con scope `test` ?
10. Perché non bisogna versionare la cartella `target` ?

Maven Avanzato

Introduzione

Dopo aver imparato a utilizzare Maven per compilare e gestire le dipendenze di un progetto, è il momento di approfondire le funzionalità avanzate che lo rendono uno strumento fondamentale nello sviluppo professionale.

In questo capitolo vedremo:

- Scope delle dipendenze
- Plugin Maven
- Multi Module Project
- Profiles
- Packaging avanzato
- Shade Plugin
- Assembly Plugin
- Deploy e distribuzione degli artefatti

Dependency Scope

Lo scope determina **quando una dipendenza è disponibile**.

Scope principali

Scope	Compilazione	Esecuzione	Test
compile	✓	✓	✓
provided	✓	✗	✓
runtime	✗	✓	✓
test	✗	✗	✓

compile

È lo scope predefinito.

```
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
  <version>9.3.0</version>  
</dependency>
```

Disponibile sempre.

provided

La libreria è disponibile in compilazione ma fornita dall'ambiente di esecuzione.

Tipico esempio:

```
<dependency>  
  <groupId>jakarta.servlet</groupId>  
  <artifactId>jakarta.servlet-api</artifactId>  
  <version>6.1.0</version>  
  <scope>provided</scope>  
</dependency>
```

Utilizzata nei server applicativi.

runtime

Necessaria solo in esecuzione.

```
<scope>runtime</scope>
```

Tipico esempio:

- driver JDBC

test

Utilizzata esclusivamente nei test.

```
<scope>test</scope>
```

Esempio:

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter</artifactId>  
  <version>5.12.2</version>  
  <scope>test</scope>  
</dependency>
```

Dipendenze Transitive

Quando aggiungiamo una libreria, Maven scarica automaticamente le sue dipendenze.

```
Applicazione
|
+-- Spring Boot
    |
    +-- Jackson
    |
    +-- Logging
    |
    +-- Validation
```

Visualizzare le dipendenze

```
mvn dependency:tree
```

Esempio:

```
progetto
|
+-- spring-boot-starter-web
   |
   +-- jackson
   +-- tomcat
```

Esclusione di dipendenze

Talvolta occorre evitare il download di una dipendenza transitiva.

```
<dependency>

  <groupId>...</groupId>

  <artifactId>...</artifactId>

  <exclusions>

    <exclusion>

      <groupId>...</groupId>

      <artifactId>...</artifactId>

    </exclusion>

    ---

  </exclusions>

</dependency>
```

Plugin Maven

I plugin estendono le funzionalità di Maven.

Maven Compiler Plugin

Compila il progetto.

```
<plugin>
  <groupId>
    org.apache.maven.plugins
  </groupId>

  <artifactId>
    maven-compiler-plugin
  </artifactId>

  <version>3.14.0</version>
  ---
</plugin>
```

Maven Surefire Plugin

Esegue i test.

```
<plugin>  
  <artifactId>  
    maven-surefire-plugin  
  </artifactId>  
</plugin>
```

Maven Javadoc Plugin

Genera la documentazione.

```
mvn javadoc:javadoc
```

Multi Module Project

Problema

Un'applicazione enterprise può contenere:

```
applicazione
|
+-- api
+-- service
+-- repository
+-- web
+-- common
```

Gestire tutto in un unico modulo diventa complesso.

Soluzione

Multi Module Project.

```
progetto-root
|
+-- pom.xml
|
+-- common
|
+-- repository
|
+-- service
|
+-- web
```

Parent POM

```
<packaging>pom</packaging>
```


Dichiarazione moduli

```
<modules>  
  <module>common</module>  
  <module>repository</module>  
  <module>service</module>  
  <module>web</module>  
</modules>
```

Build completa

```
mvn clean install
```

Compila automaticamente tutti i moduli.

Maven Profiles

Consentono configurazioni differenti.

Esempio:

- sviluppo
- test
- produzione

Definizione

```
<profiles>  
  <profile>  
    <id>dev</id>  
  </profile>  
  <profile>  
    <id>prod</id>  
  </profile>  
</profiles>
```

Attivazione

```
mvn package -Pdev
```

oppure:

```
mvn package -Pprod
```

Packaging

JAR

Applicazioni Java standard.

```
<packaging>jar</packaging>
```

WAR

Applicazioni Web.

```
<packaging>war</packaging>
```


POM

Modulo aggregatore.

```
<packaging>pom</packaging>
```

Maven Shade Plugin

Problema

L'applicazione dipende da molte librerie.

Distribuire:

```
app.jar  
lib1.jar  
lib2.jar  
lib3.jar  
...
```

non è pratico.

Soluzione

Creare un Fat Jar.

```
app-fat.jar
```

contenente:

- applicazione
- dipendenze

Configurazione

```
<plugin>  
  <groupId>  
    org.apache.maven.plugins  
  </groupId>  
  
  <artifactId>  
    maven-shade-plugin  
  </artifactId>  
  
</plugin>
```

Build

```
mvn package
```

Genera un unico JAR eseguibile.

Maven Assembly Plugin

Alternativa allo Shade Plugin.

Permette di generare:

- zip
- tar.gz
- bundle completi

Esempio

```
app/  
|  
+-- bin/  
+-- conf/  
+-- lib/
```


Maven Dependency Plugin

Analisi dipendenze.

```
mvn dependency:analyze
```

Permette di individuare:

- dipendenze inutilizzate
- dipendenze mancanti

Maven Enforcer Plugin

Garantisce regole di progetto.

Esempi:

- versione minima Java
- versione minima Maven
- dipendenze consentite

Maven Release Plugin

Gestione automatica delle release.

Tipico workflow:

```
mvn release:prepare  
mvn release:perform
```

Deploy Artefatti

Repository Locale

```
mvn install
```

Installa in:

```
~/.m2/repository
```

Repository Remoto

```
mvn deploy
```

Pubblica su:

- Nexus
- Artifactory
- GitHub Packages

Maven e CI/CD

Maven si integra facilmente con:

- Jenkins
- GitHub Actions
- GitLab CI
- Azure DevOps

Esempio pipeline:

```
git push
  |
  v
mvn clean test
  |
  v
mvn package
  |
  v
mvn deploy
```

Best Practice

Utilizzare sempre il Dependency Tree

```
mvn dependency:tree
```

Evitare dipendenze duplicate

Possono causare conflitti.

Usare Parent POM

Per centralizzare:

- versioni
- plugin
- configurazioni

Separare i moduli

Tipica struttura:

```
api  
service  
repository  
web
```

Automatizzare la build

Utilizzare pipeline CI/CD.

Errori comuni

Scope errato

Esempio:

```
<scope>test</scope>
```

su una libreria necessaria in produzione.

Dipendenze inutilizzate

Aumentano tempi di build e dimensione degli artefatti.

Fat Jar mancanti

L'applicazione funziona sul PC dello sviluppatore ma non sui server.

Versioni incompatibili

Conflitti tra librerie transitive.

Esercizi

Esercizio 1

Analizzare le dipendenze di un progetto con:

```
mvn dependency:tree
```

Esercizio 2

Aggiungere una dipendenza con scope:

```
provided
```

Esercizio 3

Creare due profili:

```
dev  
prod
```

Esercizio 4

Realizzare un Multi Module Project composto da:

```
common  
service  
web
```

Esercizio 5

Generare un Fat Jar tramite Shade Plugin.

Esercizio 6

Configurare Maven Javadoc Plugin.

Esercizio 7

Eseguire:

```
mvn dependency:analyze
```

e interpretare il risultato.

Domande di ripasso

1. Quali sono gli scope principali di Maven?
2. Cosa sono le dipendenze transitive?
3. A cosa serve `dependency:tree` ?
4. Cos'è un plugin Maven?
5. Qual è il ruolo del Parent POM?
6. Quando utilizzare un Multi Module Project?
7. A cosa servono i Profiles?
8. Qual è la differenza tra JAR e WAR?
9. Cos'è un Fat Jar?
10. Qual è la differenza tra `install` e `deploy` ?

Modularità Java

Introduzione

Con Java 9 è stato introdotto **Project Jigsaw**, un sistema di modularizzazione che consente di suddividere le applicazioni in componenti ben definiti.

Prima di Java 9 tutte le classi erano inserite nel classpath:

```
Applicazione
|
+-- Libreria A
+-- Libreria B
+-- Libreria C
```

Le dipendenze erano poco controllate e spesso si verificavano problemi di:

- conflitti tra librerie
- dipendenze nascoste
- difficoltà di manutenzione

Cos'è un Modulo

Un modulo è un insieme di:

- package
- classi
- risorse

accompagnato da un file speciale:

```
module-info.java
```

che ne descrive la struttura.

Esempio

```
com.example.app
├── module-info.java
└── com
    ├── example
    │   └── app
    │       └── Main.java
```

Il file module-info.java

È il cuore del sistema di modularizzazione.

Esempio minimo:

```
module com.example.app {  
  
}
```


Creazione di un Modulo

Supponiamo di avere:

```
package com.example.app;  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Modules");  
    }  
}
```

e:

```
module com.example.app {  
  
}
```

requires

Permette di dichiarare una dipendenza.

Esempio:

```
module com.example.app {  
    requires java.sql;  
}
```

Il modulo può utilizzare le API JDBC.

Più dipendenze

```
module com.example.app {  
    requires java.sql;  
    requires java.xml;  
}
```

Moduli della JDK

La JDK è composta da molti moduli.

Alcuni esempi:

Modulo	Contenuto
java.base	Fondamentale
java.sql	JDBC
java.xml	XML
java.desktop	Swing e AWT
java.net.http	HTTP Client

Elenco moduli

```
java --list-modules
```

Output abbreviato:

```
java.base  
java.sql  
java.xml  
java.desktop  
java.net.http
```

java.base

È il modulo fondamentale.

Contiene:

- String
- Object
- Collections
- Exceptions
- Streams

È sempre disponibile.

Non serve dichiararlo.

exports

Per impostazione predefinita i package di un modulo sono nascosti.

Per renderli pubblici:

```
module com.example.lib {  
    exports com.example.lib;  
}
```

Esempio

```
module scuola.core {  
    exports scuola.studenti;  
}
```

Le classi del package:

```
scuola.studenti
```

diventano accessibili dall'esterno.

Modulo Provider

Supponiamo di avere:

```
package scuola.studenti;  
  
public class Studente {  
  
}
```

e:

```
module scuola.core {  
  
    exports scuola.studenti;  
  
}
```

Modulo Consumer

```
module scuola.app {  
    requires scuola.core;  
}
```

Ora può utilizzare:

```
import scuola.studenti.Studente;
```

exports selettivo

È possibile esportare verso moduli specifici.

```
module scuola.core {  
    exports scuola.studenti  
        to scuola.web;  
}
```

opens

Alcuni framework utilizzano Reflection.

Ad esempio:

- Spring
- Hibernate
- Jackson

In questi casi può essere necessario usare:

```
module scuola.core {  
    opens scuola.studenti;  
}
```

Differenza tra exports e opens

exports

Permette:

```
import package
```

opens

Permette:

Reflection

Confronto

Operazione	exports	opens
Import classi	✓	✗
Reflection	✗	✓
Accesso normale	✓	✗

opens completo

Per aprire tutto il modulo:

```
open module scuola.core {  
  
}
```

Da utilizzare con cautela.

requires transitive

Permette di propagare una dipendenza.

Senza transitive

```
A --> B --> C
```

Se A usa C:

```
requires B;  
requires C;
```

Con transitive

Nel modulo B:

```
requires transitive C;
```

Nel modulo A:

```
requires B;
```

e C diventa automaticamente disponibile.

requires static

Dipendenza disponibile solo in compilazione.

```
requires static lombok;
```

Molto usato con:

- Lombok
- Annotation Processor

Service Loader

Uno dei meccanismi più potenti dei moduli.

Permette di creare plugin.

Definizione del servizio

```
public interface MessaggioService {  
    String messaggio();  
}
```

Provider

```
public class ItalianoService
    implements MessaggioService {

    @Override
    public String messaggio() {

        return "Ciao";

    }

}
```


Dichiarazione del provider

```
module provider {  
    provides MessaggioService  
        with ItalianoService;  
}
```

Utilizzo del servizio

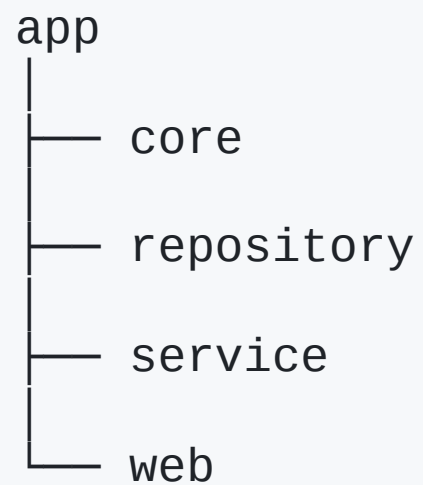
```
module consumer {  
    uses MessaggioService;  
}
```

Caricamento

```
ServiceLoader<MessaggioService> loader =  
    ServiceLoader.load(  
        MessaggioService.class);
```

Struttura di un'app modulare

```
graph TD; app --> core; app --> repository; app --> service; app --> web;
```



Esempio enterprise

```
gestione-studenti
├── common
├── model
├── dao
├── service
└── application
```

Compilazione

Directory

```
src
├── scuola.core
└── scuola.app
```

Compilare

```
javac -d out \  
--module-source-path src \  
$(find src -name "*.java")
```


Esecuzione

```
java \  
--module-path out \  
-m scuola.app/scuola.app.Main
```

Creazione JAR modulari

```
jar \  
--create \  
--file scuola.core.jar \  
-C out/scuola.core .
```

jlink

Uno dei vantaggi principali della modularità.

Permette di creare runtime personalizzati.

Esempio

```
jlink \  
--module-path mods \  
--add-modules scuola.app \  
--output runtime
```

Genera una JVM minimale.

Vantaggi

- dimensioni ridotte
- startup più veloce
- deployment semplificato

Vantaggi dei Moduli

Migliore incapsulamento

Espono solo ciò che serve.

Dipendenze esplicite

Tutte le dipendenze sono dichiarate.

Sicurezza

Riduce l'accesso indesiderato.

Runtime personalizzati

Grazie a:

jlink

Limiti dei Moduli

Compatibilità

Molte librerie legacy non sono modulari.

Complessità

Per progetti piccoli spesso non conviene.

Framework moderni

Spring Boot raramente utilizza moduli Java in produzione.

Moduli e Maven

Struttura tipica:

```
src/main/java
├── module-info.java
└── com
```

Compiler Plugin

```
<plugin>  
  <artifactId>  
    maven-compiler-plugin  
  </artifactId>  
</plugin>
```

gestisce automaticamente la compilazione modulare.

Best Practice

Modularizzare applicazioni grandi

Evitare moduli inutili

Un modulo per ogni package è eccessivo.

Usare exports minimali

Esporre solo il necessario.

Utilizzare opens solo quando serve

Specialmente con framework che usano Reflection.

Errori comuni

Dimenticare requires

Produce errori di compilazione.

Dimenticare exports

Le classi risultano invisibili.

Confondere exports e opens

Sono concetti diversi.

Modularizzare troppo presto

Spesso è meglio introdurre i moduli quando il progetto cresce.

Esercizi

Esercizio 1

Creare un modulo:

```
scuola.core
```

contenente una classe `Studente` .

Esercizio 2

Esportare il package:

```
exports scuola.studenti;
```

Esercizio 3

Creare un modulo consumer che utilizzi `Studente` .

Esercizio 4

Aggiungere una dipendenza:

```
requires java.sql;
```

Esercizio 5

Utilizzare:

```
requires transitive
```

in una catena di tre moduli.

Esercizio 6

Creare un semplice Service Provider.

Esercizio 7

Generare un runtime personalizzato tramite:

```
jlink
```

Domande di ripasso

1. Cos'è Project Jigsaw?
2. Qual è il ruolo di `module-info.java` ?
3. A cosa serve `requires` ?
4. A cosa serve `exports` ?
5. Qual è la differenza tra `exports` e `opens` ?
6. Cos'è `requires transitive` ?
7. Cos'è `requires static` ?
8. Qual è il vantaggio del Service Loader?
9. A cosa serve `jlink` ?
10. Quando conviene utilizzare i moduli Java?

Testing con JUnit

Introduzione

Lo sviluppo professionale del software non consiste soltanto nello scrivere codice funzionante, ma anche nel verificare che continui a funzionare nel tempo.

Il **testing automatico** permette di:

- individuare errori rapidamente
- prevenire regressioni
- facilitare il refactoring
- documentare il comportamento del codice
- aumentare la qualità del software

I test sono una componente essenziale delle moderne pratiche di sviluppo software.

Cos'è il Testing

Il testing consiste nell'eseguire controlli automatici sul software per verificare che il comportamento ottenuto corrisponda a quello atteso.

Principali tipologie di test

Unit Test

Verificano singole unità di codice:

- metodi
- classi
- componenti isolati

Sono veloci e semplici da eseguire.

Integration Test

Verificano l'interazione tra più componenti.

Ad esempio:

```
Service  
|  
Repository  
|  
Database
```

Functional Test

Verificano che il sistema soddisfi i requisiti funzionali richiesti dall'utente.

System Test

Testano l'intera applicazione come un sistema completo.

JUnit

JUnit è il framework standard per il testing in Java.

Permette di:

- scrivere test automatici
- eseguire test in batch
- verificare risultati attesi
- organizzare suite di test
- integrare il testing con Maven e IDE

Dipendenza Maven

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter</artifactId>  
  <version>5.12.2</version>  
  <scope>test</scope>  
</dependency>
```

Primo Test

Classe da testare:

```
public class Calcolatrice {  
    public int somma(int a, int b) {  
        return a + b;  
    }  
}
```

Classe di test

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class CalcolatriceTest {

    @Test
    void testSomma() {

        Calcolatrice c =
            new Calcolatrice();

        int risultato =
            c.somma(2,3);

        assertEquals(5, risultato);

    }

}
```

L'annotazione @Test

Indica che un metodo rappresenta un caso di test.

```
@Test  
void testSomma() {  
  
}
```

JUnit individua automaticamente tutti i metodi annotati.

Il ciclo di vita dei test

JUnit permette di eseguire codice prima e dopo ogni test.

@BeforeEach

Eseguito prima di ogni test.

```
@BeforeEach  
void init() {  
  
}
```

@AfterEach

Eseguito dopo ogni test.

```
@AfterEach  
void cleanup() {  
  
}
```

Esempio completo

```
import org.junit.jupiter.api.*;

public class DemoTest {

    @BeforeEach
    void setup() {

        System.out.println("Prima");

    }

    @Test
    void test1() {

        System.out.println("Test");

    }

    @AfterEach
    void teardown() {

        System.out.println("Dopo");

    }

}
```

Fixture

Una fixture rappresenta lo stato iniziale necessario per eseguire un test.

Esempio:

```
private Calcolatrice calcolatrice;  
  
@BeforeEach  
void setup() {  
    calcolatrice =  
        new Calcolatrice();  
}
```

Assertions

Le assertion confrontano il risultato ottenuto con quello atteso.

assertEquals

```
assertEquals(5, risultato);
```

assertNotEquals

```
assertNotEquals(10, risultato);
```


assertTrue

```
assertTrue(numero > 0);
```

assertFalse

```
assertFalse(lista.isEmpty());
```

assertNull

```
assertNull(valore);
```

assertNotNull

```
assertNotNull(valore);
```

assertSame

Verifica che due riferimenti puntino allo stesso oggetto.

```
assertSame(obj1, obj2);
```

Test delle eccezioni

JUnit permette di verificare che venga lanciata un'eccezione.

Classe:

```
public int dividi(int a, int b) {  
    return a / b;  
}
```

Test:

```
@Test  
void testDivisionePerZero() {  
    ---  
    assertThrows(  
        ArithmeticException.class,  
        () -> {  
            c.dividi(10, 0);  
        }  
    );  
}
```

Test parametrizzati

Consentono di eseguire lo stesso test con dati differenti.

Dipendenza

Inclusa in JUnit Jupiter.

Esempio

```
@ParameterizedTest  
  
@ValueSource(ints = {  
    2, 4, 6, 8  
})  
  
void testPari(int numero) {  
    assertTrue(  
        numero % 2 == 0);  
}
```

CSV Source

```
@ParameterizedTest

@CsvSource({
    "2,4",
    "3,9",
    "4,16"
})
void testQuadrato(
    int input,
    int expected)
{
    ---

    assertEquals(
        expected,
        input * input);
}
```

Display Name

Permette di assegnare una descrizione leggibile.

```
@Test  
  
@DisplayName(  
    "Verifica somma di due numeri")  
void testSomma() {  
  
}
```

Raggruppare i test

Nested Test

```
@Nested  
class TestSomma {  
  
}
```

Permette di organizzare meglio casi complessi.

Test Suite

Una suite esegue più classi di test insieme.

```
@Suite

@SelectClasses({
    CalcolatriceTest.class,
    StudenteTest.class
})
public class AllTests {

}
```

Test Driven Development (TDD)

TDD è una metodologia che prevede:

1. Scrivere il test
2. Eseguire il test (fallisce)
3. Scrivere il codice minimo
4. Eseguire il test (passa)
5. Refactoring

Ciclo Red-Green-Refactor

RED
↓
GREEN
↓
REFACTOR

Code Coverage

Misura la percentuale di codice eseguita dai test.

Strumenti

- JaCoCo
- IntelliJ Coverage
- SonarQube

Esempio

Classe A: 100%
Classe B: 80%
Classe C: 60%

Nomenclatura dei test

Buona pratica:

```
somma_DueNumeriPositivi_RitornaRisultatoCorretto()
```

oppure:

```
shouldReturnFiveWhenAddingTwoAndThree()
```

Cosa testare

È opportuno testare:

- logica di business
- algoritmi
- validazioni
- trasformazioni dati

Più una classe è utilizzata dall'esterno, maggiore dovrebbe essere la copertura dei test.

Cosa NON testare

Generalmente non si testano:

- getter e setter banali
- costanti
- codice generato automaticamente

Best Practice

Un test = una responsabilità

Ogni test dovrebbe verificare una sola funzionalità.

Test indipendenti

I test non devono dipendere dall'ordine di esecuzione.

Automatizzare tutto

I test devono essere eseguibili automaticamente.

Evitare effetti collaterali

Un test non deve modificare lo stato utilizzato da altri test.

Utilizzare fixture chiare

Preparare sempre dati prevedibili.

Errori comuni

Troppi assert nello stesso test

```
@Test
void test() {
    assertEquals(...);
    assertEquals(...);
    assertEquals(...);
}
```

Meglio dividere in test più piccoli.

Test dipendenti

```
testA();  
testB();
```

Ogni test deve essere autonomo.

Test fragili

Dipendono da:

- data corrente
- file esterni
- rete
- database reale

Nomi poco descrittivi

Evitare:

```
test1()
```

Preferire:

```
calcolaIVA_Importo100_Ritorna22()
```

Esercizi

Esercizio 1

Creare una classe:

```
Calcolatrice
```

con metodo:

```
somma(int a, int b)
```

e relativo test.

Esercizio 2

Testare:

```
sottrazione()  
moltiplicazione()  
divisione()
```

Esercizio 3

Verificare che una divisione per zero generi un'eccezione.

Esercizio 4

Creare una fixture tramite:

```
@BeforeEach
```

Esercizio 5

Realizzare un test parametrizzato.

Esercizio 6

Creare una suite contenente almeno tre classi di test.

Esercizio 7

Applicare il ciclo TDD per sviluppare una classe `Studente` .

Domande di ripasso

1. Cos'è un Unit Test?
2. Qual è il ruolo di JUnit?
3. A cosa serve l'annotazione `@Test` ?
4. Qual è la differenza tra `@BeforeEach` e `@AfterEach` ?
5. Cos'è una fixture?
6. A cosa serve `assertEquals()` ?
7. Come si verifica un'eccezione?
8. Cosa sono i test parametrizzati?
9. Cos'è il TDD?
10. Quali caratteristiche deve avere un buon test?

Mockito

Introduzione

Quando scriviamo Unit Test, spesso una classe dipende da altre classi.

Ad esempio:

```
OrdineService
|
+---- ClienteRepository
|
+---- MailService
```

Se vogliamo testare `OrdineService`, non vogliamo realmente:

- accedere al database
- inviare email
- chiamare API esterne

Vogliamo testare soltanto la logica di `OrdineService`.

Per questo si utilizzano i **Mock Object**

Cos'è Mockito

Mockito è il framework più utilizzato in Java per creare oggetti simulati (mock).

Permette di:

- simulare dipendenze
- controllare i valori restituiti
- verificare chiamate ai metodi
- isolare la logica da testare

Dipendenza Maven

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-core</artifactId>  
  <version>5.18.0</version>  
  <scope>test</scope>  
</dependency>
```

Per JUnit 5:

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-junit-jupiter</artifactId>  
  <version>5.18.0</version>  
  <scope>test</scope>  
</dependency>
```

Problema senza Mock

Supponiamo di avere:

```
public class StudenteRepository {  
    public Studente findById(int id) {  
        // accesso database  
        return null;  
    }  
}
```

e:

```
public class StudenteService {  
    private StudenteRepository repository;  
    public StudenteService(  

```

Creazione di un Mock

```
StudenteRepository repository =  
    Mockito.mock(  
        StudenteRepository.class);
```

oppure:

```
import static org.mockito.Mockito.*;  
  
StudenteRepository repository =  
    mock(StudenteRepository.class);
```


Configurare il comportamento

when(...).thenReturn(...)

```
when(repository.findById(1))  
    .thenReturn(  
        new Studente(  
            1,  
            "Mario"));
```

Quando il metodo viene chiamato, Mockito restituisce l'oggetto specificato.

Primo test con Mockito

```
@Test
void testNomeStudente() {

    StudenteRepository repository =
        mock(
            StudenteRepository.class);

    when(repository.findById(1))
        .thenReturn(
            new Studente(
                1,
                "Mario"));

    StudenteService service =
        new StudenteService(
            repository);

    String nome =
        service.getNomeStudente(1);

    assertEquals(
        "Mario",
        nome);

}
```

Mock vs Stub

Stub

Restituisce dati predefiniti.

```
when(...)  
    .thenReturn(...);
```

Mock

Permette anche di verificare il comportamento.

```
verify(...)
```

Verifica delle chiamate

Mockito può controllare che un metodo sia stato invocato.

```
verify(repository)  
    .findById(1);
```

Esempio

```
@Test
void testFindByIdInvocato() {

    repository.findById(1);

    verify(repository)
        .findById(1);

}
```


Numero di invocazioni

times()

```
verify(repository,  
        times(2))  
        .findById(1);
```

never()

```
verify(repository,  
        never())  
        .delete(1);
```

atLeast()

```
verify(repository,  
        atLeast(1))  
        .save(any());
```

Matchers

Consentono di verificare parametri variabili.

any()

```
when(repository.findById(anyInt()))  
    .thenReturn(studente);
```

anyString()

```
when(service.find(anyString()))  
    .thenReturn(...);
```

eq()

```
verify(repository)
    .findById(eq(10));
```


Simulare eccezioni

thenThrow()

```
when(repository.findById(1))  
    .thenThrow(  
        new RuntimeException());
```

Test

```
assertThrows(  
    RuntimeException.class,  
    () -> service.getNomeStudiante(1)  
);
```

Annotazioni Mockito

Mockito può creare automaticamente i mock.

@Mock

```
@Mock  
private StudentRepository repository;
```

@InjectMocks

```
@InjectMocks  
private StudentService service;
```

Mockito crea il Service e inietta i mock.

Configurazione con JUnit 5

```
@ExtendWith(MockitoExtension.class)
class StudenteServiceTest {

}
```

Esempio completo

```
@ExtendWith(MockitoExtension.class)
class StudenteServiceTest {

    @Mock
    private StudenteRepository repository;

    @InjectMocks
    private StudenteService service;

    @Test
    void testNomeStudente() {

        when(repository.findById(1))
            .thenReturn(
                new Studente(
                    1,
                    "Mario"));

        String nome =
            service.getNomeStudente(1);

        assertEquals(
            "Mario",
            nome);

        verify(repository)
            .findById(1);

    }
}
```


ArgumentCaptor

Permette di catturare gli argomenti passati a un metodo.

Esempio

```
ArgumentCaptor<Studente> captor =  
    ArgumentCaptor.forClass(  
        Studente.class);
```

Utilizzo

```
verify(repository)
    .save(captor.capture());
```

```
Studente s =
    captor.getValue();
```

Verifica

```
assertEquals(  
    "Mario",  
    s.getNome());
```

Spy

Uno Spy è un oggetto reale controllato da Mockito.

Creazione

```
List<String> lista =  
    new ArrayList<>();  
  
List<String> spy =  
    spy(lista);
```

Differenza

Mock:

```
List<String> mock =  
    mock(List.class);
```

oggetto simulato.

Spy:

```
List<String> spy =  
    spy(new ArrayList<>());
```

oggetto reale.

doReturn()

Con gli Spy è preferibile:

```
doReturn("Mario")  
    .when(spy)  
    .get(0);
```

InOrder

Verifica l'ordine delle chiamate.

```
InOrder order =  
    inOrder(  
        repository,  
        service);
```

Verifica

```
order.verify(repository)
    .save(any());

order.verify(service)
    .sendEmail();
```

Mocking di Void

Metodo:

```
public void salva() {  
  
}
```

Simulazione

```
doNothing()  
    .when(repository)  
    .save(any());
```

Eccezione

```
doThrow(  
    new RuntimeException()  
    .when(repository)  
    .save(any()));
```

Testing Service Layer

Scenario tipico:

```
Controller  
|  
Service  
|  
Repository
```

Cosa testare

Nel test del Service:

- Repository mockato
- Database escluso
- Verifica della logica

Esempio

```
when(repository.existsById(1))  
    .thenReturn(true);
```

Verifica:

```
assertTrue(  
    service.esiste(1));
```

Best Practice

Mockare solo le dipendenze

Non mockare la classe da testare.

Corretto:

```
@InjectMocks  
OrdineService service;
```

Un comportamento per test

Evitare test troppo complessi.

Verificare il risultato

Prima:

```
assertEquals(...)
```

poi:

```
verify(...)
```

Usare nomi descrittivi

```
shouldReturnStudentNameWhenIdExists()
```

Errori comuni

Mockare tutto

Produce test inutili.

Verificare dettagli interni

Meglio verificare il comportamento osservabile.

Troppi verify()

```
verify(...)  
verify(...)  
verify(...)  
verify(...)
```

Segnale di test fragile.

Non verificare il risultato

Mockito non sostituisce le assertion JUnit.

Esercizi

Esercizio 1

Creare un mock di:

```
StudenteRepository
```

e configurare:

```
findById(1)
```

Esercizio 2

Testare un `StudenteService` .

Esercizio 3

Verificare che:

```
save()
```

venga chiamato una sola volta.

Esercizio 4

Utilizzare:

```
anyInt()
```

in un test.

Esercizio 5

Simulare un'eccezione tramite:

```
thenThrow()
```

Esercizio 6

Utilizzare:

```
@Mock  
@InjectMocks
```

Esercizio 7

Utilizzare `ArgumentCaptor` per verificare l'oggetto salvato.

Domande di ripasso

1. Cos'è Mockito?
2. Qual è la differenza tra Mock e Stub?
3. A cosa serve `verify()` ?
4. Quando si usa `times()` ?
5. A cosa servono i Matchers?
6. Come si simula un'eccezione?
7. Qual è il ruolo di `@Mock` ?
8. Qual è il ruolo di `@InjectMocks` ?
9. Cos'è un `spy` ?
10. A cosa serve `ArgumentCaptor` ?

Logging

Introduzione

Durante lo sviluppo è molto comune utilizzare:

```
System.out.println("Operazione eseguita");
```

oppure:

```
System.err.println("Errore");
```

Questo approccio può andare bene per piccoli programmi didattici, ma diventa rapidamente insufficiente nelle applicazioni professionali.

In un'applicazione enterprise occorre:

- distinguere messaggi informativi ed errori
- salvare log su file
- filtrare i messaggi
- registrare timestamp

Cos'è il Logging

Il logging consiste nella registrazione degli eventi che avvengono durante l'esecuzione di un programma.

Esempi:

Utente autenticato

Ordine creato

Errore durante il salvataggio

Connessione al database fallita

Perché non usare `System.out.println()`

Problemi:

```
System.out.println("Errore");
```

- nessun livello di gravità
- nessun filtraggio
- nessuna configurazione
- difficile da gestire in produzione

Framework di Logging

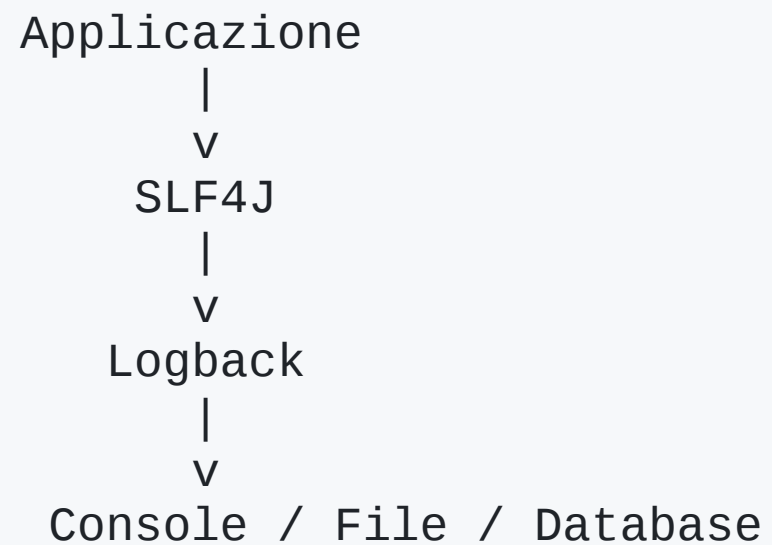
Nel mondo Java i più diffusi sono:

Framework	Ruolo
java.util.logging	incluso nella JDK
Log4j2	framework storico
SLF4J	facade standard
Logback	implementazione moderna

Oggi la combinazione più diffusa è:

SLF4J + Logback

Architettura



SLF4J

SLF4J significa:

Simple Logging Facade For Java

Non è un logger.

È un'interfaccia comune che permette di cambiare implementazione senza modificare il codice.

Dipendenze Maven

SLF4J

```
<dependency>  
  <groupId>org.slf4j</groupId>  
  <artifactId>slf4j-api</artifactId>  
  <version>2.0.17</version>  
</dependency>
```

Logback

```
<dependency>  
  <groupId>ch.qos.logback</groupId>  
  <artifactId>logback-classic</artifactId>  
  <version>1.5.18</version>  
</dependency>
```

Creazione del Logger

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Main {

    private static final Logger logger =
        LoggerFactory.getLogger(Main.class);

}
```


Primo Log

```
logger.info("Applicazione avviata");
```

Output:

```
2026-01-10 10:15:20  
INFO  
Applicazione avviata
```

Livelli di Logging

Dal meno grave al più grave:

TRACE
DEBUG
INFO
WARN
ERROR

TRACE

Livello più dettagliato.

```
logger.trace(  
    "Valore corrente {}",  
    valore);
```

Utilizzato raramente in produzione.

DEBUG

Informazioni utili durante lo sviluppo.

```
logger.debug(  
    "Utente trovato: {}",  
    username);
```

INFO

Messaggi operativi normali.

```
logger.info(  
    "Ordine creato");
```

WARN

Situazioni anomale ma non bloccanti.

```
logger.warn(  
    "Spazio disco quasi esaurito");
```

ERROR

Errori che richiedono attenzione.

```
logger.error(  
    "Errore durante il salvataggio");
```

Parametri nei Log

Evitare:

```
logger.info(  
    "Utente: " + nome);
```

Preferire:

```
logger.info(  
    "Utente: {}",  
    nome);
```


Vantaggi

- migliore performance
- codice più leggibile
- gestione automatica dei parametri

Più Parametri

```
logger.info(  
    "Utente {} ha acquistato {}",  
    username,  
    prodotto);
```

Output:

```
Utente Mario ha acquistato Notebook
```

Logging delle Eccezioni

Errore comune:

```
logger.error(  
    e.getMessage());
```

Corretto

```
logger.error(  
    "Errore durante il salvataggio",  
    e);
```

Esempio

```
try {  
    salva();  
}  
catch(Exception e) {  
    logger.error(  
        "Salvataggio fallito",  
        e);  
}
```

Configurazione Logback

File:

```
src/main/resources/logback.xml
```

Configurazione minima

```
<configuration>  
  <root level="INFO">  
    <appender-ref ref="STDOUT"/>  
  </root>  
</configuration>
```

Appender

Un Appender definisce dove scrivere i log.

Possibili destinazioni:

- Console
- File
- Database
- Socket
- Cloud Logging

Console Appender

```
<appender  
  name="STDOUT"  
  class=  
    "ch.qos.logback.core.ConsoleAppender">  
  
</appender>
```

File Appender

```
<appender  
  name="FILE"  
  class=  
    "ch.qos.logback.core.FileAppender">  
  
  <file>app.log</file>  
  
</appender>
```

Rolling File Appender

Genera file giornalieri.

```
app-2026-01-01.log  
app-2026-01-02.log  
app-2026-01-03.log
```

Configurazione

```
<rollingPolicy  
  class=  
    "ch.qos.logback.core.rolling.TimeBasedRollingPolicy">  
  
    <fileNamePattern>  
      logs/app-%d{yyyy-MM-dd}.log  
    </fileNamePattern>  
  
</rollingPolicy>
```

Pattern Layout

Personalizza il formato.

```
<pattern>  
%d{HH:mm:ss}  
[%thread]  
%-5level  
%logger  
- %msg%n  
</pattern>
```

Output

```
14:30:20  
[main]  
INFO  
Main  
- Applicazione avviata
```

Logger Specifici

Per una classe:

```
private static final Logger logger =  
    LoggerFactory.getLogger(  
        StudenteService.class);
```

Configurazione

```
<logger  
  name="it.corsojava.service"  
  level="DEBUG"/>
```


Logging e Spring Boot

Spring Boot utilizza:

SLF4J + Logback

automaticamente.

Non è necessario aggiungere dipendenze.

application.properties

Configurare il livello:

```
logging.level.root=INFO
```

Livello per package

```
logging.level.it.corsojava=DEBUG
```

Log su File

```
logging.file.name=app.log
```

Formato Log

```
logging.pattern.console=  
%d{HH:mm:ss}  
%-5level  
%logger  
- %msg%n
```

Logging Strutturato

Applicazioni moderne spesso producono log JSON.

Esempio:

```
{  
  "timestamp": "2026-01-10T10:15:20",  
  "level": "INFO",  
  "message": "Ordine creato",  
  "user": "mario"  
}
```

Facilita:

- Elastic Stack
- Kibana
- Grafana
- Splunk

MDC (Mapped Diagnostic Context)

Permette di aggiungere informazioni contestuali.

```
MDC.put(  
    "requestId",  
    "REQ123");
```


Log

```
logger.info(  
    "Ordine creato");
```

Output:

```
REQ123 Ordine creato
```

Best Practice

Non usare `System.out.println()`

Preferire sempre il logger.

Utilizzare i livelli corretti

DEBUG
INFO
WARN
ERROR

Loggare le eccezioni complete

Corretto:

```
logger.error(  
    "Errore",  
    exception);
```

Evitare informazioni sensibili

Mai scrivere:

```
password  
token  
carte di credito
```

nei log.

Utilizzare parametri

Corretto:

```
logger.info(  
    "Utente {}",  
    nome);
```

Errori comuni

Livello DEBUG in produzione

Produce file enormi.

Troppi log

Rendono difficile individuare i problemi reali.

Eccezioni senza stack trace

```
logger.error(  
    e.getMessage());
```

è insufficiente.

Informazioni sensibili nei log

Rischio di sicurezza.

Esercizi

Esercizio 1

Sostituire tutti i:

```
System.out.println()
```

con SLF4J.

Esercizio 2

Creare log di livello:

```
TRACE  
DEBUG  
INFO  
WARN  
ERROR
```

Esercizio 3

Configurare un logger che scriva su file.

Esercizio 4

Registrare una eccezione completa.

Esercizio 5

Configurare Logback con Rolling File Appender.

Esercizio 6

Configurare livelli diversi per package differenti.

Esercizio 7

Configurare il logging in un progetto Spring Boot.

Domande di ripasso

1. Cos'è il logging?
2. Perché evitare `System.out.println()` ?
3. Cos'è SLF4J?
4. Cos'è Logback?
5. Quali sono i livelli di logging?
6. A cosa serve un Appender?
7. Cos'è un Rolling File Appender?
8. Come si logga correttamente un'eccezione?
9. Cos'è l'MDC?
10. Quali informazioni non dovrebbero mai comparire nei log?

Architetture Applicative

Introduzione

Quando un'applicazione cresce, diventa difficile mantenere il codice se tutta la logica è concentrata in poche classi.

Un'applicazione professionale deve essere:

- manutenibile
- testabile
- scalabile
- estendibile

Per raggiungere questi obiettivi si utilizzano delle **architetture software**, ovvero modi organizzati di strutturare il codice.

In questo capitolo vedremo:

- Layered Architecture
- MVC

Questi concetti costituiscono la base di framework moderni come Spring Boot.

Separazione delle Responsabilità

Uno dei principi fondamentali dell'ingegneria del software è:

Separation of Concerns

Ogni componente dovrebbe avere una responsabilità specifica.

Esempio errato:

```
public class Main {  
    public static void main(String[] args) {  
        // UI  
        ---  
        // Business Logic  
        // SQL  
        // File
```

Layered Architecture

La soluzione più diffusa è l'architettura a livelli.

```
Presentation Layer
|
Business Layer
|
Persistence Layer
|
Database
```

Ogni livello comunica soltanto con il livello sottostante.

Presentation Layer

Gestisce l'interazione con l'utente.

Esempi:

- Console
- Swing
- JavaFX
- Web
- REST API

Responsabilità

- ricevere input
- mostrare output
- validare dati semplici

Non deve contenere logica di business.

Business Layer

Contiene le regole dell'applicazione.

Esempi:

Calcolo IVA

Verifica requisiti

Calcolo sconti

Responsabilità

- applicare regole aziendali
- coordinare operazioni
- gestire casi d'uso

Persistence Layer

Gestisce l'accesso ai dati.

Esempi:

- JDBC
- JPA
- Hibernate
- Spring Data

Responsabilità

- leggere dati
- salvare dati
- aggiornare dati
- eliminare dati

Esempio Completo

```
Utente
|
Controller
|
Service
|
Repository
|
Database
```

MVC

MVC significa:

Model
View
Controller

Model

Contiene i dati.

```
public class Studente {  
    private Long id;  
    private String nome;  
}
```

View

Visualizza i dati.

Esempi:

- JSP
- Thymeleaf
- Swing
- JavaFX

Controller

Riceve le richieste e coordina l'applicazione.

```
@Controller  
public class StudentController {  
  
}
```

Flusso MVC

```
Utente  
|  
View  
|  
Controller  
|  
Model
```

DAO Pattern

DAO significa:

Data Access Object

Obiettivo

Separare il codice SQL dal resto dell'applicazione.

Entity

```
public class Studente {  
    private Long id;  
    private String nome;  
}
```

DAO Interface

```
public interface StudenteDAO {  
    List<Studente> findAll();  
    Optional<Studente> findById(Long id);  
    void save(Studente studente);  
    void delete(Long id);  
}
```

Implementazione

```
public class StudenteDAOImpl  
    implements StudenteDAO {  
  
}
```

Repository Pattern

Evoluzione moderna del DAO.

Molto utilizzato in:

- Spring Data JPA
- Hibernate

Repository

```
public interface StudenteRepository {  
  
}
```

Con Spring Data

```
public interface StudenteRepository  
    extends JpaRepository<  
        Studente,  
        Long> {  
  
}
```

DAO vs Repository

DAO	Repository
JDBC	JPA
SQL esplicito	Query automatiche
Basso livello	Alto livello

Service Layer

Il Service Layer contiene la logica applicativa.

Problema

Controller troppo complesso:

```
@Controller  
public class StudenteController {  
  
    // logica  
  
}
```

Soluzione

```
@Controller  
public class StudenteController {  
    private StudenteService service;  
  
}
```

Service

```
@Service  
public class StudenteService {  
  
}
```

Flusso Corretto

```
Controller  
|  
Service  
|  
Repository
```

DTO

DTO significa:

Data Transfer Object

Problema

Esporre direttamente le Entity.

```
@Entity
public class Studente {

    private Long id;

    private String nome;

    private String password;

}
```

Soluzione

```
public class StudenteDTO {  
    private Long id;  
    private String nome;  
}
```


Benefici

- sicurezza
- disaccoppiamento
- maggiore controllo

Mapper

Converte Entity e DTO.

Entity

```
public class Studente {  
    private Long id;  
    private String nome;  
}
```

DTO

```
public class StudenteDTO {  
    private Long id;  
    private String nome;  
}
```

Mapper

```
public class StudenteMapper {  
  
    public static StudenteDTO toDTO(  
        Studente s) {  
  
        StudenteDTO dto =  
            new StudenteDTO();  
  
        dto.setId(s.getId());  
  
        dto.setNome(s.getNome());  
  
        return dto;  
  
    }  
    ---  
}
```

Dependency Injection

Una delle tecniche più importanti nello sviluppo moderno.

Problema

```
public class StudenteService {  
    private StudenteRepository repository =  
        new StudenteRepository();  
}
```

La dipendenza è creata internamente.

Svantaggi

- forte accoppiamento
- difficile da testare
- difficile da sostituire

Constructor Injection

```
public class StudenteService {  
  
    private final  
    StudenteRepository repository;  
  
    public StudenteService(  
        StudenteRepository repository) {  
  
        this.repository =  
            repository;  
  
    }  
  
}
```

Vantaggi

- testabilità
- flessibilità
- disaccoppiamento

Inversion of Control

Con Spring:

```
@Service  
public class StudenteService {  
  
}
```

Il framework crea e collega automaticamente gli oggetti.

```
Applicazione  
  |  
Spring Container  
  |  
Bean
```

Architettura Spring Boot

Struttura tipica:

```
src/main/java
|
+-- controller
|
+-- service
|
+-- repository
|
+-- dto
|
+-- mapper
|
+-- entity
|
+-- config
```

Esempio Completo

```
StudenteController
|
StudenteService
|
StudenteRepository
|
Database
```

Architettura a Pacchetti

Piccoli progetti:

```
controller  
service  
repository  
dto  
entity
```

Architettura per Feature

Progetti grandi:

```
studente
|
+-- controller
+-- service
+-- repository
+-- dto

corso
|
+-- controller
+-- service
+-- repository
+-- dto
```


Architettura Monolitica

Tutti i moduli nella stessa applicazione.

```
Applicazione
|
+-- Utenti
+-- Corsi
+-- Docenti
```

Vantaggi

- semplicità
- deployment facile

Svantaggi

- crescita complessa

Microservizi

Ogni dominio è un'applicazione indipendente.

Servizio Utenti

Servizio Corsi

Servizio Pagamenti

Vantaggi

- scalabilità
- indipendenza

Svantaggi

- maggiore complessità

Best Practice

Controller sottili

Il controller deve delegare al service.

Service con logica

La business logic appartiene al service.

Repository per accesso dati

Mai SQL nel controller.

Utilizzare DTO

Non esporre direttamente le entity.

Constructor Injection

È la forma consigliata di Dependency Injection.

Errori Comuni

SQL nel Controller

```
@Controller  
public class StudenteController {  
  
    // query SQL  
  
}
```

Logica nel Repository

Il repository deve gestire solo i dati.

Entity esposte nelle API

Meglio utilizzare DTO.

Dipendenze create con new

```
new Repository()
```

Meglio utilizzare Dependency Injection.

Esercizi

Esercizio 1

Progettare una Layered Architecture per una gestione studenti.

Esercizio 2

Creare:

StudenteDAO

e relativa implementazione.

Esercizio 3

Creare:

```
StudenteRepository
```

Esercizio 4

Creare:

StudenteService

Esercizio 5

Creare:

StudenteDT0

e relativo Mapper.

Esercizio 6

Applicare Constructor Injection.

Esercizio 7

Disegnare l'architettura completa:

```
Controller
|
Service
|
Repository
|
Database
```

Domande di ripasso

1. Cos'è la Separation of Concerns?
2. Quali livelli compongono una Layered Architecture?
3. Cosa significa MVC?
4. Qual è il ruolo del Controller?
5. Qual è il ruolo del Service?
6. Qual è il ruolo del Repository?
7. Cos'è un DTO?
8. Perché utilizzare un Mapper?
9. Cos'è la Dependency Injection?
10. Quali vantaggi offre la Constructor Injection?

Spring Core

Introduzione

Spring Framework è il framework Java più utilizzato nello sviluppo enterprise.

Nasce con l'obiettivo di semplificare la realizzazione di applicazioni:

- modulari
- testabili
- manutenibili
- scalabili

I concetti fondamentali di Spring sono:

- Inversion of Control (IoC)
- Dependency Injection (DI)
- Bean
- Container

Tutto il resto di Spring Boot, Spring MVC e Spring Data si basa su questi concetti.

Perché Spring?

Prima di Spring era comune scrivere codice come:

```
public class StudenteService {  
    private StudenteRepository repository =  
        new StudenteRepository();  
}
```

Problemi:

- forte accoppiamento
- difficile da testare
- difficile da sostituire
- difficile da configurare

Inversion of Control (IoC)

Approccio tradizionale

L'applicazione crea gli oggetti.

```
StudenteRepository repository =  
    new StudenteRepository();
```


Con IoC

È il framework che crea e gestisce gli oggetti.

```
Applicazione
  |
  v
Spring Container
  |
  v
Bean
```

Vantaggi

- minore accoppiamento
- maggiore riuso
- migliore testabilità
- configurazione centralizzata

Dependency Injection

La Dependency Injection è l'implementazione pratica dell'IoC.

Esempio

```
public class StudenteService {  
  
    private final  
    StudenteRepository repository;  
  
    public StudenteService(  
        StudenteRepository repository) {  
  
        this.repository = repository;  
  
    }  
  
}
```

Il Service non crea il Repository.

Lo riceve dall'esterno.

Bean

Un Bean è un oggetto gestito dal container Spring.

Classe normale

```
public class StudenteService {  
  
}
```

Bean Spring

```
@Service  
public class StudenteService {  
  
}
```

Spring crea automaticamente l'istanza.

Spring Container

Il Container è il componente che:

- crea i bean
- li collega
- li configura
- ne gestisce il ciclo di vita

Implementazione principale

`ApplicationContext`

ApplicationContext

È il contenitore centrale di Spring.

```
ApplicationContext context;
```

Recuperare un Bean

```
StudenteService service =  
    context.getBean(  
        StudenteService.class);
```

Normalmente nelle applicazioni moderne non si utilizza direttamente.

Spring esegue l'iniezione automaticamente.

Configurazione tramite Annotazioni

La modalità moderna utilizza annotazioni.

@Component

Annotazione base.

```
@Component  
public class LoggerService {  
  
}
```

@Service

Indica un componente di business logic.

```
@Service  
public class StudenteService {  
  
}
```

@Repository

Indica un componente di accesso ai dati.

```
@Repository  
public class StudentRepository {  
  
}
```


@Controller

Indica un controller MVC.

```
@Controller  
public class StudenteController {  
  
}
```

Relazione tra annotazioni

```
@Component
|
+-- @Service
|
+-- @Repository
|
+-- @Controller
```

Tutte derivano da `@Component` .

Component Scan

Spring deve individuare automaticamente i bean.

Configurazione

@ComponentScan

Esempio

```
@Configuration  
  
@ComponentScan(  
    "it.corsojava")  
public class AppConfig {  
  
}
```

Spring analizzerà il package.

Constructor Injection

Metodo consigliato.

Repository

```
@Repository  
public class StudenteRepository {  
  
}
```

Service

```
@Service
public class StudenteService {

    private final
    StudenteRepository repository;

    public StudenteService(
        StudenteRepository repository) {

        this.repository = repository;

    }

}
```


Spring individua automaticamente la dipendenza.

Field Injection

Possibile ma sconsigliata.

```
@Autowired  
private StudentRepository repository;
```

Problemi

- difficile da testare
- dipendenze nascoste
- minore leggibilità

Setter Injection

Alternativa meno comune.

```
@Autowired
public void setRepository(
    StudenteRepository repository) {

    this.repository = repository;
}
```

Quale scegliere?

Tipo	Consigliato
Constructor Injection	✓
Setter Injection	△
Field Injection	✗

Configurazione Java-Based

Non sempre è possibile utilizzare annotazioni.

Classe di configurazione

```
@Configuration  
public class AppConfig {  
  
}
```

Metodo @Bean

```
@Bean
public StudenteRepository repository() {
    return new StudenteRepository();
}
```


Esempio Completo

```
@Configuration
public class AppConfig {

    @Bean
    public StudenteRepository repository() {

        return new StudenteRepository();

    }

    @Bean
    public StudenteService service() {

        return new StudenteService(
            repository());

    }

}
```

Recuperare Bean dal Context

```
AnnotationConfigApplicationContext  
    context =  
    new AnnotationConfigApplicationContext(  
        AppConfig.class);
```

Ottenere il Bean

```
StudenteService service =  
    context.getBean(  
        StudenteService.class);
```

Scope dei Bean

Per default:

singleton

Singleton

Una sola istanza.

Container

|
+ - - - - Bean

Prototype

Nuova istanza ogni volta.

```
@Scope("prototype")
```

Lifecycle dei Bean

Spring gestisce:

```
Creazione
|
Inizializzazione
|
Utilizzo
|
Distruzione
```

@PostConstruct

Eseguito dopo la creazione.

```
@PostConstruct  
public void init() {  
  
}
```


@PreDestroy

Eseguito prima della distruzione.

```
@PreDestroy  
public void cleanup() {  
  
}
```

Qualifier

Problema:

```
@Repository  
class MySqlConnection {}  
  
@Repository  
class OracleRepository {}
```

Spring trova due bean compatibili.

Errore:

```
NoUniqueBeanDefinitionException
```

Soluzione

```
@Qualifier("mySqlRepository")
```

Esempio

```
@Service
public class StudenteService {

    public StudenteService(

        @Qualifier(
            "mySqlRepository")

        StudenteRepository repository) {

    }

}
```

Primary

Permette di definire un bean predefinito.

```
@Primary  
@Repository  
public class MySqlRepository {  
  
}
```

Eventi Spring

Il container può pubblicare eventi.

Evento

```
public class StudenteCreatoEvent {  
  
}
```


Listener

```
@EventListener  
public void gestisciEvento(  
    StudenteCreatoEvent e) {  
  
}
```

Environment

Permette di leggere proprietà.

```
@Autowired  
private Environment env;
```

Lettura

```
String nome =  
    env.getProperty(  
        "app.nome");
```

Properties

File:

```
application.properties
```

Esempio

```
app.nome=Gestione  Studenti
```

Value

Alternativa più semplice.

```
@Value("${app.nome}")  
private String nome;
```

Architettura Tipica

```
Controller  
|  
Service  
|  
Repository
```

Tutti gestiti dal container Spring.

Best Practice

Utilizzare Constructor Injection

Sempre la scelta preferita.

Usare annotazioni specifiche

```
@Service  
@Repository  
@Controller
```

anziché il generico:

```
@Component
```

Evitare Field Injection

Riduce la testabilità.

Utilizzare Bean Singleton

Per la maggior parte dei casi.

Centralizzare la configurazione

In:

```
@Configuration
```

Errori Comuni

Utilizzare new

```
new StudentRepository()
```

aggira il container Spring.

Dipendenze circolari

A -> B

B -> A

da evitare.

Troppe responsabilità in un Bean

Un bean dovrebbe avere un unico scopo.

Package fuori dal Component Scan

Spring non troverà i bean.

Esercizi

Esercizio 1

Creare un bean:

```
@Service  
public class CorsoService {  
  
}
```

Esercizio 2

Creare un Repository e iniettarlo tramite costruttore.

Esercizio 3

Configurare un bean tramite:

```
@Bean
```

Esercizio 4

Utilizzare:

```
@Value
```

per leggere una proprietà.

Esercizio 5

Creare due implementazioni dello stesso repository e utilizzare:

```
@Qualifier
```


Esercizio 6

Utilizzare:

```
@PostConstruct
```

e:

```
@PreDestroy
```

Esercizio 7

Realizzare una piccola applicazione composta da:

```
StudenteController  
    |  
StudenteService  
    |  
StudenteRepository
```

interamente gestita dal container Spring.

Domande di ripasso

1. Cos'è Spring Framework?
2. Cos'è l'Inversion of Control?
3. Cos'è la Dependency Injection?
4. Cos'è un Bean?
5. Qual è il ruolo dell'ApplicationContext?
6. Qual è la differenza tra `@Component` e `@Service` ?
7. Perché la Constructor Injection è preferibile?
8. A cosa serve `@Bean` ?
9. Qual è lo scope predefinito dei bean?
10. A cosa servono `@Qualifier` e `@Primary` ?

Approfondimento su @Autowired

Dichiarare `final` la proprietà `repository` in questo contesto non è solo una buona pratica, ma è il modo standard e più pulito per gestire la **Dependency Injection** in Spring (tramite costruttore).

Ecco i motivi principali per cui dovresti farlo:

1. Garantisce l'Immutabilità

Una volta che Spring ha avviato l'applicazione e ha iniettato `StudenteRepository` nel costruttore, la variabile `repository` non può più essere modificata. Nessuno, all'interno della classe `StudenteService`, potrà accidentalmente riassegnarla (ad esempio scrivendo `this.repository = null;` o istanziando un altro oggetto). Questo rende il codice più sicuro e meno incline a bug improvvisi a runtime.

2. Thread Safety (Sicurezza nei thread)

In Spring, i service come `@Service` sono di default dei **Singleton** (esiste una sola istanza della classe in tutta l'applicazione). Poiché l'applicazione gestisce più richieste contemporaneamente su thread diversi, avere proprietà `final` (e quindi immutabili) garantisce che la classe sia intrinsecamente *thread-safe*. Non rischierai mai che un thread modifichi la dipendenza di un altro thread.

3. Ti "costringe" a inizializzarla subito

Il compilatore Java è molto severo con la parola chiave `final`. Se dichiari una variabile `final`, ti obbliga a inizializzarla o direttamente sulla linea di dichiarazione o **all'interno del costruttore**. Questo ti assicura che non esisterà mai un'istanza di `StudenteService` con un `repository` a `null`.

Un piccolo "trucco" per pulire il codice

Visto che stai usando la Dependency Injection basata su costruttore (che è la più raccomandata rispetto a `@Autowired` sui campi), se nel tuo progetto usi **Lombok**, puoi sfruttare `@RequiredArgsConstructor`.

Il tuo codice diventerebbe incredibilmente compatto, così:

```
@Service
@RequiredArgsConstructor // Genera automaticamente il costruttore per tutte le proprietà final
public class StudenteService {

    private final StudenteRepository repository;

    // Il costruttore non serve più scriverlo a mano!
}
```


In sintesi: mettere `final` ti protegge da errori di distrazione, rende l'applicazione più robusta ed esplicita chiaramente che quella dipendenza è un pilastro fondamentale e non modificabile del tuo servizio.

Spring Boot

Introduzione

Spring Boot è un framework costruito sopra Spring Framework che semplifica enormemente la creazione di applicazioni Java moderne.

Prima di Spring Boot era necessario configurare manualmente:

- Bean
- Dispatcher Servlet
- Tomcat
- XML di configurazione
- Dependency Management

Con Spring Boot molte configurazioni vengono effettuate automaticamente.

Obiettivo:

Convention over Configuration

Scrivere meno configurazione e più logica applicativa.

Cos'è Spring Boot

Spring Boot è una piattaforma che permette di creare rapidamente:

- API REST
- Applicazioni Web
- Microservizi
- Applicazioni Enterprise
- Batch Application

Vantaggi

- configurazione minima
- server integrato
- dependency management
- auto configurazione
- produzione rapida

Architettura

```
Applicazione
|
Spring Boot
|
Spring Framework
|
Tomcat
|
JVM
```

Spring Boot Starter

Uno Starter è un insieme di dipendenze preconfigurate.

Invece di aggiungere molte librerie:

```
spring-core  
spring-context  
spring-web  
spring-mvc  
jackson  
...
```


si utilizza:

```
spring-boot-starter-web
```

Starter più comuni

Starter	Funzione
spring-boot-starter	Core
spring-boot-starter-web	REST e Web
spring-boot-starter-data-jpa	JPA
spring-boot-starter-test	Testing
spring-boot-starter-security	Security
spring-boot-starter-validation	Validation

Creazione del Progetto

Utilizzando:

- Spring Initializr
- IntelliJ IDEA
- Eclipse
- VS Code

Maven

```
<parent>

  <groupId>
    org.springframework.boot
  </groupId>

  <artifactId>
    spring-boot-starter-parent
  </artifactId>

  <version>3.5.0</version>

</parent>
```

Dipendenza Web

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-starter-web  
    </artifactId>  
  
</dependency>
```

Classe Principale

Ogni applicazione Spring Boot ha una classe principale.

```
@SpringBootApplication
public class Application {

    public static void main(
        String[] args) {

        SpringApplication.run(
            Application.class,
            args);

    }
}
```

@SpringBootApplication

È una meta-annotazione.

Equivale a:

`@Configuration`

`@EnableAutoConfiguration`

`@ComponentScan`

Auto Configuration

Uno dei punti di forza di Spring Boot.

Senza Spring Boot

Configurazione manuale.

```
@Bean  
public ObjectMapper objectMapper() {  
  
}
```

Con Spring Boot

```
spring-boot-starter-web
```

configura automaticamente:

- Jackson
- DispatcherServlet
- Tomcat
- Conversion Service
- Validation

Embedded Server

Spring Boot integra un server web.

Di default:

Tomcat

Vantaggio

Non serve installare un server esterno.

Avvio

```
mvn spring-boot:run
```

Oppure

```
java -jar app.jar
```

Prima Applicazione

```
@SpringBootApplication
public class DemoApplication {

    public static void main(
        String[] args) {

        SpringApplication.run(
            DemoApplication.class,
            args);

    }

}
```

Primo Controller REST

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {

        return "Hello Spring Boot";

    }

}
```


Risultato

```
http://localhost:8080/hello
```

Output:

```
Hello Spring Boot
```

Struttura Tipica

```
src
|
+-- main
|   |
|   +-- java
|   |
|   +-- resources
|
+-- test
```

Package Consigliati

```
controller  
service  
repository  
entity  
dto  
mapper  
config
```

application.properties

File principale di configurazione.

Posizione:

```
src/main/resources
```

Porta

```
server.port=8081
```

Nome applicazione

```
spring.application.name=scuola
```

Livello log

```
logging.level.root=INFO
```

application.yml

Alternativa moderna.

```
server:  
  port: 8081  
  
spring:  
  application:  
    name: scuola
```


Lettura Proprietà

@Value

```
@Value("${server.port}")  
private String porta;
```

ConfigurationProperties

Metodo consigliato.

```
@ConfigurationProperties(  
    prefix = "app")  
public class AppConfig {  
    private String nome;  
  
}
```

CommandLineRunner

Permette di eseguire codice all'avvio.

```
@Component
public class StartupRunner
    implements CommandLineRunner {

    @Override
    public void run(
        String... args)
        throws Exception {

        System.out.println(
            "Applicazione avviata");

    }

}
```

ApplicationRunner

Alternativa più moderna.

```
@Component
public class StartupRunner
    implements ApplicationRunner {

    @Override
    public void run(
        ApplicationArguments args)
        throws Exception {

        ---

    }
}
```

Bean Configuration

```
@Configuration
public class AppConfiguration {

    @Bean
    public CorsoService service() {

        return new CorsoService();

    }

}
```

REST API

Spring Boot nasce soprattutto per creare API.

GET

```
@GetMapping("/studenti")
```


POST

```
@PostMapping("/studenti")
```

PUT

```
@PutMapping("/studenti/{id}")
```

DELETE

```
@DeleteMapping("/studenti/{id}")
```

JSON Automatico

Classe:

```
public class Studente {  
    private Long id;  
    private String nome;  
}
```

Controller

```
@GetMapping("/studente")  
public Studente getStudente() {  
  
    return new Studente(  
        1L,  
        "Mario");  
  
}
```

Output

```
{  
  "id": 1,  
  "nome": "Mario"  
}
```

Jackson esegue automaticamente la serializzazione.

Packaging

Creazione JAR eseguibile.

```
mvn package
```

Output

```
target/
```


Avvio

```
java -jar applicazione.jar
```

DevTools

Riavvio automatico durante lo sviluppo.

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-devtools  
    </artifactId>  
  
</dependency>
```

Actuator

Monitoraggio applicazione.

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-starter-actuator  
    </artifactId>  
  
</dependency>
```

Endpoint

```
/actuator/health
```

```
/actuator/info
```

```
/actuator/metrics
```

Spring Boot e Testing

Starter dedicato:

```
<dependency>

    <groupId>
        org.springframework.boot
    </groupId>

    <artifactId>
        spring-boot-starter-test
    </artifactId>

    <scope>test</scope>
    - - -
</dependency>
```

Best Practice

Utilizzare Starter ufficiali

Constructor Injection

Sempre preferibile.

Controller sottili

La logica va nei Service.

Configurare tramite Properties

Mai hardcodare valori.

Utilizzare Actuator

Per monitoraggio e diagnostica.

Errori Comuni

Troppe logiche nel Controller

```
@RestController  
public class StudenteController {  
  
    // business logic  
  
}
```

Utilizzare Field Injection

```
@Autowired  
private Service service;
```

Configurazione sparsa

Meglio centralizzare in:

```
application.properties
```

Utilizzare `System.out.println`

Preferire sempre il logging.

Esercizi

Esercizio 1

Creare un progetto Spring Boot con:

```
spring-boot-starter-web
```

Esercizio 2

Realizzare un endpoint:

```
GET /hello
```

Esercizio 3

Configurare una porta diversa da:

8080

Esercizio 4

Creare una classe `Studente` e restituirla come JSON.

Esercizio 5

Implementare un `CommandLineRunner` .

Esercizio 6

Aggiungere Spring Boot Actuator.

Esercizio 7

Realizzare una piccola API REST:

```
GET studenti  
POST studenti  
PUT studenti  
DELETE studenti
```


Domande di ripasso

1. Cos'è Spring Boot?
2. Quali vantaggi offre rispetto a Spring?
3. Cos'è uno Starter?
4. A cosa serve `@SpringBootApplication` ?
5. Cos'è l'Auto Configuration?
6. Quale server embedded utilizza Spring Boot?
7. A cosa serve `application.properties` ?
8. Qual è la differenza tra `CommandLineRunner` e `ApplicationRunner` ?
9. Cos'è Actuator?
10. Come si genera un JAR eseguibile?

Spring MVC

Introduzione

Spring MVC è il framework web di Spring che implementa il pattern:

```
Model  
View  
Controller
```

Permette di sviluppare:

- applicazioni web tradizionali
- portali aziendali
- dashboard
- applicazioni CRUD
- API REST

Spring Boot integra automaticamente Spring MVC tramite:

```
spring-boot-starter-web
```

Il Pattern MVC

MVC separa l'applicazione in tre componenti.

```
Utente  
  |  
  v  
Controller  
  |  
  v  
Model  
  |  
  v  
View
```

Model

Rappresenta i dati.

```
public class Studente {  
    private Long id;  
    private String nome;  
}
```

View

Visualizza le informazioni.

Esempi:

- HTML
- Thymeleaf
- JSP

Controller

Riceve le richieste HTTP e coordina il flusso.

```
@Controller  
public class StudenteController {  
  
}
```


DispatcherServlet

È il cuore di Spring MVC.

```
Browser
|
DispatcherServlet
|
Controller
|
Service
```

Tutte le richieste passano attraverso il DispatcherServlet.

Primo Controller

```
@Controller
public class HomeController {

    @GetMapping("/")

    public String home() {

        return "home";

    }

}
```

Request Mapping

Permette di associare URL e metodi.

GetMapping

```
@GetMapping("/studenti")  
public String elenco() {  
    return "studenti";  
}
```

PostMapping

```
@PostMapping("/studenti")
```

PutMapping

```
@PutMapping("/studenti/{id}")
```

DeleteMapping

```
@DeleteMapping("/studenti/{id}")
```


RequestMapping

Forma generica.

```
@RequestMapping(  
    value="/studenti",  
    method=RequestMethod.GET)
```

Parametri della Richiesta

RequestParam

URL:

```
/cerca?nome=Mario
```

Controller:

```
@GetMapping("/cerca")
public String cerca(

    @RequestParam
    String nome) {

    ---

    return nome;

}
```

Parametro opzionale

```
@RequestParam(  
    required = false)
```

Valore di default

```
@RequestParam(  
    defaultValue = "anonimo")
```

PathVariable

URL:

```
/studenti/10
```

Controller

```
@GetMapping(  
    "/studenti/{id}")  
public String trova(  
    @PathVariable  
    Long id) {  
    return "ID = " + id;  
}
```

Più Variabili

```
@GetMapping(  
    "/corsi/{idCorso}/studenti/{id}")
```



```
public String metodo(  
    @PathVariable Long idCorso,  
    @PathVariable Long id)  
{  
    ...  
}
```

Restituire una View

```
@GetMapping("/")  
public String home() {  
    return "home";  
}
```

Spring cerca:

```
templates/home.html
```

Il Model

Serve a passare dati alla View.

Esempio

```
@GetMapping("/")  
public String home(  
    Model model) {  
  
    model.addAttribute(  
        "nome",  
        "Mario");  
  
    return "home";  
  
}
```

Thymeleaf

```
<h1 th:text="${nome}">  
</h1>
```

Output:

```
<h1>Mario</h1>
```

Thymeleaf

È il motore di template più utilizzato con Spring.

Dipendenza:

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-starter-thymeleaf  
    </artifactId>  
    ---  
</dependency>
```

Cicli

```
<tr th:each="s : ${studenti}">
```

Controller

```
@GetMapping("/studenti")
public String elenco(
    Model model) {

    model.addAttribute(
        "studenti",
        lista);

    return "studenti";

}
```


Form HTML

Pagina:

```
<form action="/studenti"  
      method="post">  
  
  <input name="nome">  
  
  <button>  
    Salva  
  </button>  
  
</form>
```

Ricevere il Form

```
@PostMapping("/studenti")
public String salva(

    @RequestParam
    String nome)

{
    ...
}
```

Binding Automatico

Classe:

```
public class StudenteDTO {  
    private String nome;  
  
}
```

Controller

```
@PostMapping("/studenti")  
public String salva(  
    StudenteDTO dto) {  
  
    ...  
}
```

Spring valorizza automaticamente l'oggetto.

Redirect

```
return "redirect:/studenti";
```

Forward

```
return "forward:/studenti";
```

Gestione Errori

```
@ExceptionHandler(  
    Exception.class)
```

Esempio

```
@ExceptionHandler(  
    RuntimeException.class)  
  
public String errore() {  
    return "errore";  
}
```


REST Controller

Quando si sviluppano API REST.

Controller tradizionale

```
@Controller
```

Restituisce View.

REST Controller

```
@RestController
```

Restituisce dati.

Esempio REST

```
@RestController
public class StudenteController {

    @GetMapping("/studente")

    public Studente getStudente() {

        return new Studente(
            1L,
            "Mario");
    }
}
```

Output JSON

```
{  
  "id": 1,  
  "nome": "Mario"  
}
```

RequestBody

Ricevere JSON.

JSON

```
{  
  "nome": "Mario"  
}
```

Controller

```
@PostMapping("/studenti")  
  
public void salva(  
    @RequestBody  
    StudenteDTO dto)  
  
{  
}
```


ResponseEntity

Permette di controllare la risposta HTTP.

Esempio

```
@GetMapping("/studenti/{id}")  
  
public ResponseEntity<Studente>  
    trova(  
        @PathVariable Long id)  
{  
    return ResponseEntity.ok(  
        studente);  
}
```

Codici HTTP

Codice	Significato
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Restituire Created

```
return ResponseEntity  
    .status(201)  
    .body(dto);
```

Controller + Service

Architettura corretta:

```
Controller  
  |  
Service  
  |  
Repository
```

Esempio

```
@RestController  
  
public class StudenteController {  
    private final  
    StudenteService service;  
  
}
```

Validazione

```
@PostMapping("/studenti")
public void salva(

    @Valid
    @RequestBody
    StudenteDTO dto)

{
}
```

Logging nel Controller

```
logger.info(  
    "Richiesta ricevuta");
```


Best Practice

Controller sottili

La logica va nei Service.

DTO nelle API

Mai esporre direttamente le Entity.

Utilizzare ResponseEntity

Per controllare lo status HTTP.

Utilizzare @Valid

Per validare gli input.

Gestire le eccezioni

Centralmente tramite:

```
@ControllerAdvice
```

Errori Comuni

SQL nel Controller

```
@RestController  
  
public class Controller {  
    // query SQL  
  
}
```


Logica di Business nel Controller

Meglio nel Service.

Entity come Input API

Preferire DTO.

Nessuna validazione

Può causare dati inconsistenti.

Esercizi

Esercizio 1

Creare un controller:

```
@GetMapping("/hello")
```

Esercizio 2

Leggere un parametro:

```
@RequestParam
```

Esercizio 3

Leggere una variabile:

```
@PathVariable
```

Esercizio 4

Creare una View Thymeleaf.

Esercizio 5

Creare un form HTML e ricevere i dati.

Esercizio 6

Realizzare una API REST CRUD per `Studente` .

Esercizio 7

Utilizzare:

`ResponseEntity`

per gestire gli status HTTP.

Domande di ripasso

1. Cos'è Spring MVC?
2. Qual è il ruolo del DispatcherServlet?
3. Qual è la differenza tra `@Controller` e `@RestController` ?
4. A cosa serve `@GetMapping` ?
5. Qual è la differenza tra `@RequestParam` e `@PathVariable` ?
6. A cosa serve il Model?
7. Cos'è Thymeleaf?
8. A cosa serve `@RequestBody` ?
9. Perché usare `ResponseEntity` ?
10. Perché la logica di business non dovrebbe stare nel Controller?

Spring Data JPA

Introduzione

Nelle applicazioni enterprise moderne raramente si utilizza JDBC direttamente.

La soluzione più diffusa è:

```
Spring Data JPA
      |
Hibernate
      |
Database
```

Spring Data JPA riduce enormemente il codice necessario per accedere ai dati.

Con poche righe è possibile:

- salvare entità
- effettuare ricerche
- aggiornare dati
- eliminare record
- gestire paginazione e ordinamento

Cos'è JPA

JPA significa:

Java Persistence API

Non è un framework.

È una specifica standard Java.

Definisce:

- Entity
- Relazioni
- Query
- Persistenza

Hibernate

Hibernate è l'implementazione JPA più utilizzata.

```
JPA  
|  
Hibernate
```

Spring Boot utilizza Hibernate automaticamente.

Dipendenze Maven

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-starter-data-jpa  
    </artifactId>  
  
</dependency>
```

Driver Database

MySQL:

```
<dependency>  
    <groupId>  
        com.mysql  
    </groupId>  
  
    <artifactId>  
        mysql-connector-j  
    </artifactId>  
  
</dependency>
```

Configurazione

```
spring.datasource.url=  
jdbc:mysql://localhost:3306/scuola  
  
spring.datasource.username=root  
  
spring.datasource.password=password
```

Configurazione Hibernate

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

ddl-auto

Valori principali:

Valore	Significato
none	nessuna modifica
validate	verifica schema
update	aggiorna schema
create	ricrea schema
create-drop	elimina alla chiusura

Entity

Una Entity rappresenta una tabella del database.

Esempio

```
@Entity
@Table(name = "studenti")
public class Studente {

    @Id

    private Long id;

}
```


Chiave Primaria

```
@Id  
private Long id;
```

Generazione Automatica

```
@Id  
  
@GeneratedValue(  
    strategy =  
    GenerationType.IDENTITY)  
  
private Long id;
```

Campi

```
@Column(nullable = false)  
private String nome;
```

Entity Completa

```
@Entity

@Table(name="studenti")
public class Studente {

    @Id

    @GeneratedValue(
        strategy =
        GenerationType.IDENTITY)

    private Long id;

    private String nome;

    private String cognome;

}
```

Repository

Il Repository sostituisce il DAO tradizionale.

JpaRepository

```
public interface  
StudentRepository  
  
extends JpaRepository<  
    Studente,  
    Long> {  
  
}
```

Operazioni Disponibili

Automaticamente disponibili:

```
save()
```

```
findById()
```

```
findAll()
```

```
delete()
```

```
count()
```

```
existsById()
```

Salvataggio

```
Studente s =  
    new Studente();  
  
s.setNome("Mario");  
  
repository.save(s);
```


Ricerca per ID

```
Optional<Studiante> studente =  
    repository.findById(1L);
```

Ricerca di Tutti

```
List<Studente> lista =  
    repository.findAll();
```

Eliminazione

```
repository.deleteById(1L);
```

Service Layer

```
@Service
public class StudenteService {

    private final
    StudenteRepository repository;

    public StudenteService(
        StudenteRepository repository) {

        this.repository = repository;

    }

}
```

Query Methods

Uno dei punti più potenti di Spring Data.

Metodo

```
List<Studente>  
findByNome(String nome);
```

Spring genera automaticamente la query.

Esempio SQL Generato

```
SELECT *  
FROM studenti  
WHERE nome = ?
```

Ricerca per Cognome

```
List<Studente>  
findByCognome(  
    String cognome);
```


Ricerca Multipla

```
List<Studente>  
findByNomeAndCognome(  
    String nome,  
    String cognome);
```

Like

```
List<Studente>  
findByNomeContaining(  
    String testo);
```

SQL

```
WHERE nome LIKE '%testo%'
```

Ordinamento

```
List<Studente>  
findByNomeOrderByCognome();
```

Count

```
long countByNome(  
    String nome);
```

Exists

```
boolean existsByNome(  
    String nome);
```

Query Personalizzate

Quando il nome del metodo non basta.

@Query

```
@Query(  
    "SELECT s  
    FROM Studente s  
    WHERE s.nome = :nome")
```


Metodo

```
List<Studente>  
trovaPerNome(  
    @Param("nome")  
    String nome);
```

JPQL

JPA utilizza:

JPQL

Differenza

SQL:

```
SELECT *  
FROM studenti
```

JPQL:

```
SELECT s  
FROM Studente s
```

Si lavora sulle Entity.

Native Query

Possibile utilizzare SQL puro.

```
@Query(  
    value =  
    "SELECT * FROM studenti",  
    nativeQuery = true)
```

Paging

Necessario per grandi quantità di dati.

Pageable

```
Pageable pageable =  
    PageRequest.of(  
        0,  
        10);
```

Query

```
Page<Studente> page =  
    repository.findAll(  
        pageable);
```


Informazioni

```
page.getContent();
```

```
page.getTotalPages();
```

```
page.getTotalElements();
```

Sorting

```
Sort sort =  
    Sort.by("nome");
```

Ordinamento Desc

```
Sort sort =  
    Sort.by(  
        Direction.DESC,  
        "nome");
```

Relazioni

Le Entity possono essere collegate.

One To One

```
@OneToOne  
private CartaStudiante carta;
```

One To Many

```
@OneToMany  
private List<Esame> esami;
```

Many To One

```
@ManyToOne  
private Corso corso;
```

Many To Many

```
@ManyToMany  
private List<Docente> docenti;
```


Fetch Type

Determina quando caricare i dati.

Eager

```
FetchType.EAGER
```

Carica subito.

Lazy

```
FetchType.LAZY
```

Carica solo quando necessario.

Transactional

Gestione automatica delle transazioni.

```
@Transactional  
public void salva() {  
  
}
```

Dirty Checking

Hibernate rileva automaticamente le modifiche.

Esempio

```
Studente s =  
    repository.findById(1L)  
        .orElseThrow();  
  
s.setNome("Mario");
```

Hibernate aggiorna il database automaticamente.

Auditing

Memorizzazione automatica delle date.

```
@CreatedDate
```

```
private LocalDateTime createdAt;
```

@LastModifiedDate

```
private LocalDateTime updatedAt;
```


Controller

```
@RestController  
  
@RequestMapping(  
    "/studenti")  
public class StudenteController {  
  
}
```

Endpoint

```
@GetMapping  
public List<Studente>  
findAll() {  
    return service.findAll();  
}
```

Architettura Completa

```
Controller
|
Service
|
Repository
|
Hibernate
|
Database
```

Best Practice

Utilizzare Repository

Evitare DAO JDBC manuali.

Utilizzare DTO

Non esporre Entity direttamente.

Constructor Injection

Sempre preferibile.

Utilizzare Pageable

Per liste molto grandi.

Preferire FetchType.LAZY

Quando possibile.

Errori Comuni

Entity Restituite Direttamente

Possono esporre dati indesiderati.

Troppe Relazioni EAGER

Generano query enormi.

Logica nel Repository

La business logic appartiene ai Service.

Utilizzare create in Produzione

```
ddl-auto=create
```

può distruggere i dati.

Esercizi

Esercizio 1

Creare una Entity:

Studente

Esercizio 2

Creare:

```
StudenteRepository
```

estendendo `JpaRepository` .

Esercizio 3

Implementare:

```
findByNome()
```

Esercizio 4

Implementare:

```
findByNomeContaining()
```

Esercizio 5

Utilizzare:

Pageable

Esercizio 6

Creare una relazione:

@OneToMany

tra `Studente` ed `Esame` .

Esercizio 7

Realizzare una API REST completa con:

```
Controller  
Service  
Repository  
Entity  
DTO
```

Domande di ripasso

1. Cos'è JPA?
2. Qual è il ruolo di Hibernate?
3. Cos'è una Entity?
4. A cosa serve `@Id` ?
5. Qual è il ruolo di `JpaRepository` ?
6. Cosa sono i Query Methods?
7. Qual è la differenza tra SQL e JPQL?
8. A cosa serve `Pageable` ?
9. Qual è la differenza tra EAGER e LAZY?
10. A cosa serve `@Transactional` ?

REST API

Introduzione

Le moderne applicazioni software comunicano principalmente tramite **API REST**.

Esempi:

- applicazioni web
- applicazioni mobile
- microservizi
- integrazioni tra sistemi aziendali
- applicazioni cloud

REST è oggi lo standard de facto per la realizzazione di servizi distribuiti.

Cos'è una API

API significa:

Application Programming Interface

È un insieme di regole che consente a due applicazioni di comunicare.

Esempio:

```
App Mobile
  |
  v
REST API
  |
  v
Database
```

Cos'è REST

REST significa:

Representational State Transfer

È uno stile architetturale definito da:

Roy Fielding

nel 2000.

Principi REST

Una API REST dovrebbe essere:

- client-server
- stateless
- cacheable
- uniforme
- scalabile

Stateless

Ogni richiesta deve contenere tutte le informazioni necessarie.

Corretto

```
GET /studenti/10  
Authorization: Bearer token
```

Errato

Server ricorda stato precedente

Risorse

REST è basato sul concetto di risorsa.

Esempi:

```
/studenti
```

```
/docenti
```

```
/corsi
```


/esami

URI

Ogni risorsa è identificata da un URI.

Esempio:

```
/studenti/10
```

Significato

Studente con ID 10

Naming Convention

Utilizzare sostantivi.

Corretto:

```
/studenti
```

```
/corsi
```

Errato:

```
/getStudenti
```

/creaStudiante

HTTP Methods

REST utilizza i verbi HTTP.

GET

Recupera dati.

```
GET /studenti
```

Recupero singolo

```
GET /studenti/10
```

POST

Crea una nuova risorsa.

```
POST /studenti
```

Body

```
{  
  "nome": "Mario",  
  "cognome": "Rossi"  
}
```

PUT

Aggiorna completamente una risorsa.

```
PUT /studenti/10
```

PATCH

Aggiornamento parziale.

```
PATCH /studenti/10
```

DELETE

Elimina una risorsa.

```
DELETE /studenti/10
```

CRUD e REST

CRUD	HTTP
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

JSON

È il formato più utilizzato nelle API REST.

Oggetto

```
{  
  "id": 1,  
  "nome": "Mario"  
}
```

Array

```
[  
  {  
    "id":1,  
    "nome":"Mario"  
  },  
  {  
    "id":2,  
    "nome":"Anna"  
  }  
]
```

Content-Type

Indica il formato del contenuto.

Content-Type:
application/json

Accept

Indica il formato desiderato.

```
Accept:  
application/json
```

REST con Spring Boot

Controller:

```
@RestController  
  
@RequestMapping("/studenti")  
public class StudenteController {  
  
}
```

GET All

```
@GetMapping  
public List<StudenteDTO>  
findAll() {  
    return service.findAll();  
}
```

GET By Id

```
@GetMapping("/{id}")
public StudenteDTO trova(
    @PathVariable Long id)
{
    return service.findById(id);
}
```


POST

```
@PostMapping
public StudenteDTO salva(

    @RequestBody
    StudenteDTO dto)

{
    return service.save(dto);
}
```

PUT

```
@PutMapping("/{id}")
public StudenteDTO aggiorna(

    @PathVariable Long id,

    @RequestBody
    StudenteDTO dto)

{
    return service.update(
        id,
        dto);
}
```

DELETE

```
@DeleteMapping("/{id}")  
public void elimina(  
    @PathVariable Long id)  
{  
    service.delete(id);  
}
```

DTO

Le API dovrebbero esporre DTO.

Entity

```
@Entity
public class Studente {

    private Long id;

    private String password;

}
```

DTO

```
public class StudenteDTO {  
    private Long id;  
    private String nome;  
}
```

Status Code HTTP

Le API REST devono restituire codici appropriati.

200 OK

Richiesta completata correttamente.

200 OK

201 Created

Risorsa creata.

201 Created

204 No Content

Operazione eseguita senza contenuto.

204 No Content

400 Bad Request

Richiesta errata.

400 Bad Request

401 Unauthorized

Autenticazione mancante.

401 Unauthorized

403 Forbidden

Accesso negato.

403 Forbidden

404 Not Found

Risorsa inesistente.

404 Not Found

500 Internal Server Error

Errore lato server.

500 Internal Server Error

ResponseEntity

Permette di controllare la risposta.

Esempio

```
@GetMapping("/{id}")  
  
public ResponseEntity<StudenteDTO>  
trova(  
    @PathVariable Long id)  
{  
    return ResponseEntity.ok(  
        dto);  
}
```

Risorsa Creata

```
return ResponseEntity  
    .status(201)  
    .body(dto);
```

Gestione Errori

Approccio centralizzato.

ControllerAdvice

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
}
```

Not Found

```
@ExceptionHandler(  
    NotFoundException.class)
```

Esempio

```
@ExceptionHandler(  
    NotFoundException.class)  
  
public ResponseEntity<String>  
errore() {  
  
    return ResponseEntity  
        .status(404)  
        .body("Non trovato");  
  
}
```

Validazione

DTO

```
public class StudenteDTO {  
    @NotBlank  
    private String nome;  
  
}
```


Controller

```
@PostMapping  
  
public StudenteDTO salva(  
    @Valid  
    @RequestBody  
    StudenteDTO dto)  
  
{  
    return service.save(dto);  
}
```

Versioning

Le API evolvono nel tempo.

URL Versioning

```
/api/v1/studenti
```

```
/api/v2/studenti
```

Header Versioning

API-Version: 1

Paginazione

Mai restituire migliaia di record.

Endpoint

```
GET /studenti?page=0&size=20
```

Spring

```
@GetMapping  
  
public Page<StudenteDTO>  
findAll(  
    Pageable pageable)  
{  
    return service.findAll(  
        pageable);  
}
```


Ordinamento

```
GET /studenti  
?sort=nome,asc
```

Filtri

```
GET /studenti  
?nome=Mario
```

HATEOAS

Approccio avanzato REST.

Esempio

```
{
  "id":1,
  "nome":"Mario",
  "_links": {
    "self": {
      "href":
        "/studenti/1"
    }
  }
}
```

Idempotenza

Una richiesta è idempotente se produce sempre lo stesso risultato.

Metodo	Idempotente
GET	✓
PUT	✓
DELETE	✓
POST	✗

Documentazione API

Strumento più diffuso:

OpenAPI

Swagger

Permette di generare documentazione interattiva.

```
/swagger-ui.html
```

Testing API

Strumenti comuni:

- Postman
- Insomnia
- curl
- Swagger UI

Curl

```
curl http://localhost:8080/studenti
```

Architettura REST Completa

```
Client
|
REST Controller
|
Service
|
Repository
|
Database
```

Best Practice

Utilizzare nomi al plurale

```
/studenti
```

Utilizzare correttamente gli HTTP Methods

Restituire Status Code appropriati

Utilizzare DTO

Mai esporre Entity.

Gestire gli errori centralmente

Con:

```
@RestControllerAdvice
```


Implementare paginazione

Per endpoint che restituiscono liste.

Errori Comuni

Verbi negli URL

Errato:

```
/getStudenti
```

Sempre 200 OK

Anche in caso di errore.

Nessuna validazione

Entity esposte direttamente

Nessuna documentazione API

w# Esercizi

Esercizio 1

Realizzare CRUD REST completo per `Studente` .

Esercizio 2

Utilizzare `ResponseEntity`.

Esercizio 3

Implementare validazione con:

```
@NotBlank
```

Esercizio 4

Creare un `GlobalExceptionHandler` .

Esercizio 5

Implementare paginazione tramite `Pageable` .

Esercizio 6

Versionare una API con:

```
/api/v1
```

Esercizio 7

Documentare la API tramite Swagger/OpenAPI.

Domande di ripasso

1. Cosa significa REST?
2. Cos'è una risorsa REST?
3. Qual è la differenza tra GET e POST?
4. Quando utilizzare PUT e PATCH?
5. Perché utilizzare DTO?
6. Qual è il significato di 201 Created?
7. Cos'è `ResponseEntity` ?
8. A cosa serve `@Valid` ?
9. Perché implementare la paginazione?
10. Cos'è OpenAPI?

Validazione

Introduzione

Una delle regole fondamentali nello sviluppo software è:

```
Mai fidarsi dei dati ricevuti.
```

I dati possono provenire da:

- form web
- API REST
- file CSV
- database
- applicazioni esterne

La **validazione** serve a garantire che i dati rispettino determinati vincoli prima di essere elaborati.

Esempi:

- nome obbligatorio
- email valida
- età maggiore di 18 anni
- password di lunghezza minima

Nelle applicazioni Spring Boot la validazione viene gestita tramite:

Jakarta Bean Validation

implementata generalmente da:

Hibernate Validator

Perché Validare

Supponiamo di ricevere:

```
{  
  "nome": "",  
  "email": "abc",  
  "eta": -10  
}
```

Senza validazione:

- dati inconsistenti
- errori applicativi
- problemi di sicurezza
- bug difficili da individuare

Bean Validation

Bean Validation è una specifica Java che consente di dichiarare vincoli direttamente sui campi.

Esempio:

```
@NotBlank  
private String nome;
```

Dipendenza Maven

Con Spring Boot:

```
<dependency>  
    <groupId>  
        org.springframework.boot  
    </groupId>  
  
    <artifactId>  
        spring-boot-starter-validation  
    </artifactId>  
  
</dependency>
```

DTO e Validazione

Normalmente la validazione viene applicata ai DTO.

DTO

```
public class StudenteDTO {  
    private String nome;  
    private String email;  
}
```

DTO Validato

```
public class StudenteDTO {  
  
    @NotBlank  
    private String nome;  
  
    @Email  
    private String email;  
  
}
```

@NotNull

Verifica che il valore non sia:

```
null
```

Esempio

```
@NotNull  
private String nome;
```

Valido:

```
"Mario"
```

Non valido:

```
null
```

@NotEmpty

Verifica che una stringa o collezione:

- non sia null
- non sia vuota

Esempio

```
@NotEmpty  
private String nome;
```

Non valido:

```
""
```

@NotBlank

È il più utilizzato per le stringhe.

Verifica che:

- non sia null
- non sia vuota
- non contenga solo spazi

Esempio

```
@NotBlank  
private String nome;
```

Non valido:

```
""
```


" "

Confronto

Vincolo	Null	Vuoto	Spazi
NotNull	✗	✓	✓
NotEmpty	✗	✗	✓
NotBlank	✗	✗	✗

@Size

Controlla la lunghezza.

Esempio

```
@Size(  
    min = 2,  
    max = 50)  
private String nome;
```

Valido

```
"Mario"
```

Non valido

"A"

@Email

Verifica il formato email.

Esempio

```
@Email  
private String email;
```


Valido

mario@azienda.it

Non valido

```
mario.it
```

Vincoli Numerici

@Min

```
@Min(18)  
private int eta;
```

@Max

```
@Max(120)  
private int eta;
```

Esempio

```
@Min(18)  
  
@Max(120)  
  
private int eta;
```

@Positive

```
@Positive  
private double importo;
```

@PositiveOrZero

```
@PositiveOrZero  
private double saldo;
```


@Negative

```
@Negative  
private int temperatura;
```

Validazione Date

@Past

```
@Past  
private LocalDate nascita;
```

@Future

```
@Future  
private LocalDate scadenza;
```

Regex

@Pattern

```
@Pattern(  
    regexp =  
    "[A-Z]{3}[0-9]{3}")  
private String codice;
```

Valido

ABC123

Non valido

abc123

Messaggi Personalizzati

```
@NotBlank(  
    message =  
    "Il nome è obbligatorio")  
private String nome;
```

DTO Completo

```
public class StudenteDTO {  
  
    @NotBlank(  
        message =  
        "Nome obbligatorio")  
    private String nome;  
  
    @Email(  
        message =  
        "Email non valida")  
    private String email;  
  
    @Min(  
        value = 18,  
        message =  
        "Età minima 18")  
    private int eta;  
  
}
```

Attivare la Validazione

Nel Controller:

```
@PostMapping  
  
public StudenteDTO salva(  
    @Valid  
    @RequestBody  
    StudenteDTO dto)  
  
{  
    return service.save(dto);  
}
```

@Valid

Attiva la validazione automatica.

Senza @Valid

```
@RequestBody  
StudenteDTO dto
```

nessun controllo.

Con @Valid

```
@Valid  
@RequestBody  
StudenteDTO dto
```

validazione automatica.

Errore di Validazione

Input:

```
{  
  "nome": "",  
  "email": "abc"  
}
```

Risposta

```
{  
  "timestamp": "...",  
  "status": 400,  
  "error": "Bad Request"  
}
```


BindingResult

Permette di leggere gli errori.

Esempio

```
@PostMapping
```

```
public ResponseEntity<?> salva(
```

```
    @Valid
```

```
    @RequestBody
```

```
    StudenteDTO dto,
```

```
    BindingResult result)
```

```
{  
}
```

Verifica

```
if(result.hasErrors()) {  
  
}
```

Elenco Errori

```
result.getFieldErrors()
```

Gestione Centralizzata

Approccio consigliato.

ControllerAdvice

```
@RestControllerAdvice  
public class ValidationHandler {  
  
}
```

Exception Handler

```
@ExceptionHandler(  
    MethodArgumentNotValidException.class)
```

Esempio

```
@ExceptionHandler(  
    MethodArgumentNotValidException.class)  
  
public ResponseEntity<?> handle(  
    MethodArgumentNotValidException ex)  
{  
}
```


Costruzione Errori

```
Map<String,String> errors =  
    new HashMap<>();
```

Lettura

```
for(FieldError error :  
    ex.getBindingResult()  
        .getFieldErrors())  
{  
    errors.put(  
        error.getField(),  
        error.getDefaultMessage());  
}
```

Risposta

```
return ResponseEntity  
    .badRequest()  
    .body(errors);
```

Output

```
{  
  "nome":  
    "Nome obbligatorio",  
  
  "email":  
    "Email non valida"  
}
```

Validazione Annidata

DTO

```
public class CorsoDTO {  
    @Valid  
    private DocenteDTO docente;  
  
}
```

DTO Figlio

```
public class DocenteDTO {  
    @NotBlank  
    private String nome;  
  
}
```

Gruppi di Validazione

Consentono regole diverse.

Marker Interface

```
public interface CreateGroup {  
  
}
```

```
public interface UpdateGroup {  
  
}
```

Utilizzo

```
@NotNull(  
    groups = UpdateGroup.class)
```

Programmatic Validation

Validazione manuale.

Validator

```
Validator validator;
```

Esempio

```
Set<ConstraintViolation<  
    StudenteDTO>>  
errori =  
validator.validate(dto);
```

Validazione Service Layer

La validazione non dovrebbe limitarsi ai Controller.

Esempio

```
@Service  
  
@Validated  
public class StudenteService {  
  
}
```


Validazione Parametri

```
public void salva(  
    @NotBlank  
    String nome)  
  
{  
}
```

Best Practice

Validare sempre gli input

Utilizzare DTO

Non validare direttamente le Entity.

Centralizzare la gestione errori

Con:

```
@RestControllerAdvice
```

Utilizzare messaggi chiari

Validare anche nel Service Layer

Errori Comuni

DimENTICARE @Valid

```
@RequestBody  
StudenteDTO dto
```

Validare solo lato frontend

La validazione server è sempre necessaria.

Entity utilizzate come DTO

Problemi di sicurezza e manutenzione.

Messaggi generici

```
Validation failed
```

Meglio messaggi specifici.

Esercizi

Esercizio 1

Creare un DTO con:

```
@NotBlank  
@Email  
@Min
```

Esercizio 2

Validare un endpoint REST tramite:

```
@Valid
```

Esercizio 3

Creare messaggi personalizzati.

Esercizio 4

Utilizzare:

```
@Pattern
```

per validare un codice fiscale semplificato.

Esercizio 5

Gestire gli errori tramite:

```
@RestControllerAdvice
```

Esercizio 6

Implementare validazione annidata.

Esercizio 7

Applicare la validazione nel Service Layer.

Domande di ripasso

1. Cos'è Bean Validation?
2. Qual è il ruolo di Hibernate Validator?
3. Differenza tra `@NotNull`, `@NotEmpty` e `@NotBlank` ?
4. A cosa serve `@Size` ?
5. A cosa serve `@Email` ?
6. Perché utilizzare `@Valid` ?
7. Cos'è `BindingResult` ?
8. A cosa serve `@RestControllerAdvice` ?
9. Cos'è la validazione annidata?
10. Perché validare anche nel Service Layer?

Gestione Errori in Spring

Introduzione

Gli errori sono inevitabili in qualsiasi applicazione software.

Esempi:

- risorsa non trovata
- dati non validi
- errore database
- accesso non autorizzato
- errori di programmazione

Una buona applicazione deve:

- intercettare gli errori
- registrarli nei log
- restituire messaggi comprensibili
- evitare stack trace verso il client

Spring Boot offre un sistema molto potente per la gestione centralizzata delle eccezioni.

Cos'è una Eccezione

Un'eccezione è un oggetto che rappresenta una situazione anomala.

```
throw new RuntimeException(  
    "Errore");
```

Gerarchia delle Eccezioni

```
Throwable
|
+-- Error
|
+-- Exception
    |
    +-- RuntimeException
```

Checked Exception

Devono essere gestite obbligatoriamente.

```
IOException  
SQLException
```

Esempio

```
try {  
  
}  
catch(IOException e) {  
  
}
```

Runtime Exception

Non obbligano alla gestione.

`NullPointerException`

`IllegalArgumentException`

`IllegalStateException`

Esempio

```
throw new IllegalArgumentException(  
    "Parametro non valido");
```

Gestione Tradizionale

```
@GetMapping("/{id}")  
  
public StudenteDTO trova(  
    @PathVariable Long id)  
{  
    try {  
        return service.findById(id);  
    }  
    catch(Exception e) {  
        return null;  
    }  
}
```

Problemi

- codice duplicato
- difficile manutenzione
- logica sparsa

Soluzione Spring

Gestione centralizzata.

```
Controller
|
Exception
|
ControllerAdvice
```

Custom Exception

Creare eccezioni specifiche migliora la leggibilità.

Esempio

```
public class
StudenteNotFoundException
extends RuntimeException {

    public StudenteNotFoundException(
        Long id)
    {
        super(
            "Studente " +
            id +
            " non trovato");
    }
    ---
}
```

Utilizzo nel Service

```
public StudenteDTO findById(  
    Long id)  
{  
    return repository  
        .findById(id)  
        .map(mapper::toDTO)  
        .orElseThrow(  
            () ->  
                new StudenteNotFoundException(  
                    id));  
}
```

@ExceptionHandler

Permette di gestire una specifica eccezione.

Esempio

```
@ExceptionHandler(  
    StudenteNotFoundException.class)  
  
public ResponseEntity<String>  
handle(  
    StudenteNotFoundException ex)  
{  
    return ResponseEntity  
        .status(404)  
        .body(  
            ex.getMessage());  
}
```

Controller Locale

```
@RestController  
public class StudenteController {  
  
}
```

Handler

```
@ExceptionHandler(  
    RuntimeException.class)
```

Valido solo per quel controller.

@ControllerAdvice

Permette la gestione globale.

Classe

```
@ControllerAdvice  
public class GlobalExceptionHandler {  
  
}
```

@RestControllerAdvice

Versione consigliata per API REST.

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
}
```

Gestione Not Found

```
@ExceptionHandler(  
    StudenteNotFoundException.class)  
  
public ResponseEntity<String>  
handleNotFound(  
    StudenteNotFoundException ex)  
{  
    return ResponseEntity  
        .status(404)  
        .body(  
            ex.getMessage());  
}
```

Gestione IllegalArgumentException

```
@ExceptionHandler(  
    IllegalArgumentException.class)  
  
public ResponseEntity<String>  
handleValidation(  
    IllegalArgumentException ex)  
{  
    return ResponseEntity  
        .badRequest()  
        .body(  
            ex.getMessage());  
}
```

Gestione Generica

Ultima difesa.

```
@ExceptionHandler(  
    Exception.class)  
  
public ResponseEntity<String>  
handleGeneric(  
    Exception ex)  
{  
    return ResponseEntity  
        .status(500)  
        .body(  
            "Errore interno");  
}
```

Error Response Standard

Evitare:

"Studente non trovato"

Preferire

```
{  
  "timestamp":  
    "2026-01-01T10:00:00",  
  
  "status":404,  
  
  "error":  
    "Not Found",  
  
  "message":  
    "Studente 10 non trovato"  
}
```


DTO di Errore

```
public class ErrorResponse {  
    private LocalDateTime  
        timestamp;  
  
    private int status;  
  
    private String error;  
  
    private String message;  
  
}
```

Utilizzo

```
ErrorResponse error =  
    new ErrorResponse(  
        LocalDateTime.now(),  
        404,  
        "Not Found",  
        ex.getMessage());
```

Restituzione

```
return ResponseEntity  
    .status(404)  
    .body(error);
```

Gestione Errori di Validazione

Quando utilizziamo:

```
@Valid
```

Spring genera:

```
MethodArgumentNotValidException
```

Handler

```
@ExceptionHandler(  
    MethodArgumentNotValidException.class)
```

Esempio

```
public ResponseEntity<Map<
    String,
    String>>
handleValidation(
    MethodArgumentNotValidException ex)
{
    Map<String, String>
        errors =
        new HashMap<>();

    ex.getBindingResult()
        .getFieldErrors()
        .forEach(error ->

        errors.put(
            error.getField(),
            error.getDefaultMessage()));

    return ResponseEntity
        .badRequest()
        .body(errors);
}
```

Output

```
{  
  "nome":  
    "Nome obbligatorio",  
  
  "email":  
    "Email non valida"  
}
```

Logging delle Eccezioni

Ogni errore dovrebbe essere registrato.

Logger

```
private static final  
Logger logger =  
LoggerFactory.getLogger(  
    GlobalExceptionHandler.class);
```

Esempio

```
logger.error(  
    ex.getMessage(),  
    ex);
```

Exception Translation

Trasformare eccezioni tecniche in errori applicativi.

Evitare

SQLException

verso il client.

Preferire

`StudenteNotFoundException`

oppure:

`BusinessException`

Business Exception

```
public class
BusinessException
extends RuntimeException {

    public BusinessException(
        String message)
    {
        super(message);
    }

}
```

Esempio

```
if(eta < 18) {  
    throw new BusinessException(  
        "Età minima 18 anni");  
}
```

HTTP Status

Spring offre la classe:

```
HttpStatus
```


Esempio

```
return ResponseEntity  
    .status(  
        HttpStatus.NOT_FOUND)  
    .build();
```

@ResponseStatus

Alternativa semplice.

Eccezione

```
@ResponseStatus(  
    HttpStatus.NOT_FOUND)  
  
public class  
StudenteNotFoundException  
extends RuntimeException {  
  
}
```

Risultato

Automaticamente:

404 Not Found

Problem Details

Dalle versioni moderne di Spring Boot è disponibile il supporto allo standard:

RFC 9457

(Problem Details for HTTP APIs)

Classe ProblemDetail

```
ProblemDetail detail =  
    ProblemDetail  
        .forStatus(404);
```

Messaggio

```
detail.setDetail(  
    "Studente non trovato");
```

Risposta

```
return detail;
```


Output

```
{  
  "type": "about:blank",  
  "title": "Not Found",  
  "status": 404,  
  "detail":  
    "Studente non trovato"  
}
```

Gestione Globale Moderna

```
@RestControllerAdvice  
public class  
GlobalExceptionHandler {  
  
}
```

Not Found

```
@ExceptionHandler(  
    StudentNotFoundException.class)
```

Validation

```
@ExceptionHandler(  
    MethodArgumentNotValidException.class)
```

Generic

```
@ExceptionHandler(  
    Exception.class)
```

Flusso Completo

```
Client
|
Controller
|
Service
|
Exception
|
GlobalExceptionHandler
|
HTTP Response
```

Best Practice

Utilizzare Custom Exception

Gestire gli errori centralmente

Con:

```
@RestControllerAdvice
```

Loggare sempre gli errori

Restituire DTO di errore

Non esporre stack trace

Mai verso il client.

Utilizzare ProblemDetail

Nelle applicazioni moderne.

Errori Comuni

Catch generici ovunque

```
catch(Exception e)
```

sparsi nell'applicazione.

Restituire null

```
return null;
```


Nessun logging

Stack Trace esposti

Problema di sicurezza.

Sempre HTTP 200

Anche in caso di errore.

Esercizi

Esercizio 1

Creare una:

```
StudenteNotFoundException
```

Esercizio 2

Implementare:

`BusinessException`

Esercizio 3

Creare un:

```
GlobalExceptionHandler
```

Esercizio 4

Gestire:

```
MethodArgumentNotValidException
```


Esercizio 5

Creare un DTO:

ErrorResponse

Esercizio 6

Utilizzare:

```
ProblemDetail
```

Esercizio 7

Implementare la gestione completa degli errori in una API CRUD Studenti.

Domande di ripasso

1. Qual è la differenza tra Checked e Runtime Exception?
2. Perché utilizzare Custom Exception?
3. A cosa serve `@ExceptionHandler` ?
4. Qual è il ruolo di `@RestControllerAdvice` ?
5. Come si gestiscono gli errori di validazione?
6. Perché non bisogna esporre stack trace?
7. Cos'è `ProblemDetail` ?
8. A cosa serve `@ResponseStatus` ?
9. Perché utilizzare DTO di errore?
10. Quali errori dovrebbero essere loggati?

Spring Security - Fondamenti

Introduzione

Quando un'applicazione viene pubblicata su Internet, uno dei primi problemi da affrontare è la sicurezza.

Occorre stabilire:

- chi può accedere
- cosa può fare
- quali risorse può utilizzare
- come proteggere dati e API

Per questo motivo Spring Boot integra il framework:

Spring Security

Spring Security è lo standard de facto per la sicurezza nelle applicazioni Java enterprise.

Cos'è Spring Security

Spring Security è un framework che gestisce:

- autenticazione
- autorizzazione
- protezione URL
- protezione API REST
- gestione password
- sessioni
- login
- logout
- OAuth2
- JWT

Concetti Fondamentali

Prima di studiare Spring Security è necessario distinguere due concetti.

Autenticazione

Risponde alla domanda:

```
Chi sei?
```

Esempio:

```
Username: mario  
Password: *****
```

Se le credenziali sono corrette:

```
Utente autenticato
```

Autorizzazione

Risponde alla domanda:

Cosa puoi fare?

Esempio:

ADMIN

può:

- creare utenti
- eliminare utenti
- modificare dati

USER

può solo:

- consultare dati

Flusso Generale

```
Richiesta  
  |  
Autenticazione  
  |  
Autorizzazione  
  |  
Controller
```

Dipendenza Maven

Spring Boot Starter:

```
<dependency>
    <groupId>
        org.springframework.boot
    </groupId>
    <artifactId>
        spring-boot-starter-security
    </artifactId>
</dependency>
```

Primo Effetto

Dopo aver aggiunto la dipendenza:

```
spring-boot-starter-security
```

tutte le URL diventano protette automaticamente.

Avvio Applicazione

Console:

```
Using generated security password:  
8f6f...
```

Spring crea automaticamente:

```
utente: user  
password: generata casualmente
```


Login Automatico

Accedendo a:

```
http://localhost:8080
```

compare una pagina di login.

Architettura

```
Browser  
|  
Security Filter Chain  
|  
Controller
```

Security Filter Chain

È il cuore di Spring Security.

Ogni richiesta passa attraverso una serie di filtri.

```
Request
|
Filtro 1
|
Filtro 2
|
Filtro 3
|
Controller
```

Configurazione Moderna

Da Spring Security 6 si utilizza:

```
SecurityFilterChain
```

Configurazione Base

```
@Configuration

public class SecurityConfig {

    @Bean
    public SecurityFilterChain
    filterChain(
        HttpSecurity http)
        throws Exception
    {

        return http.build();

    }

}
```

Consentire Accesso Pubblico

```
@Bean
public SecurityFilterChain
filterChain(
    HttpSecurity http)
throws Exception {

    http

    .authorizeHttpRequests(auth ->
        auth

            .requestMatchers(
                "/public/**")
            .permitAll()
        ---
            .anyRequest()
            .authenticated()
    );

    return http.build();
}
```

URL Protette

```
/public/info
```

accessibile a tutti.

/admin

richiede autenticazione.

Ruoli

I ruoli rappresentano categorie di utenti.

Esempio

```
ROLE_USER
```

ROLE_ADMIN

Accesso per Ruolo

```
.requestMatchers("/admin/**")  
.hasRole("ADMIN")
```

Accesso Utenti

```
.requestMatchers("/studenti/**")  
.hasAnyRole(  
    "USER",  
    "ADMIN")
```

Utenti In Memory

Metodo semplice per i test.

Configurazione

```
@Bean
public UserDetailsService
users() {

    UserDetails admin =

        User.builder()

            .username("admin")

            .password("{noop}admin")

            .roles("ADMIN")

            .build();

    return
        new InMemoryUserDetailsManager(
            admin);

}
```

UserDetails

Rappresenta un utente autenticabile.

Campi Principali

```
username  
password  
authorities
```

Password Encoder

Mai salvare password in chiaro.

Errato

```
password123
```

Corretto

```
$2a$10...
```

(password hashata)

BCrypt

Encoder più utilizzato.

Bean

```
@Bean
public PasswordEncoder
passwordEncoder() {

    return new BCryptPasswordEncoder();

}
```

Utilizzo

```
String hash =  
passwordEncoder.encode(  
    "password123");
```

Verifica

```
passwordEncoder.matches(  
    passwordInserita,  
    passwordSalvata);
```


Database Authentication

Scenario reale:

Utente
|
Database

Entity Utente

```
@Entity
public class Utente {

    private String username;

    private String password;

}
```

Repository

```
public interface
UtenteRepository
extends JpaRepository<
    Utente,
    Long>
{
    Optional<Utente>
    findByUsername(
        String username);
}
```

UserDetailsService

Spring utilizza questa interfaccia per recuperare utenti.

Implementazione

```
@Service
public class
CustomUserDetailsService

implements UserDetailsService {

}
```

Metodo

```
@Override
```

```
public UserDetails  
loadUserByUsername(  
    String username)  
{  
}
```

Esempio

```
Utente utente =  
    repository.findByUsername(  
        username)  
    .orElseThrow(  
        UsernameNotFoundException::new);
```

Creazione UserDetails

```
return User.builder()  
    .username(  
        utente.getUsername()  
    )  
    .password(  
        utente.getPassword()  
    )  
    .roles("USER")  
    .build();
```


Login Form

Spring Security può generare automaticamente la pagina di login.

Oppure

Pagina personalizzata.

```
.formLogin(form ->  
    form.loginPage("/login"))
```

Logout

Configurazione standard:

```
.logout(logout ->  
        logout.permitAll())
```

CSRF

Cross Site Request Forgery.

Protezione automatica di Spring.

Form

```
<input  
  type="hidden"  
  name="_csrf"  
  value="...">
```

API REST

Spesso nelle API REST il CSRF viene disabilitato.

```
http.csrf(  
    csrf -> csrf.disable());
```

Basic Authentication

Metodo semplice.

Header HTTP

```
Authorization:  
Basic base64(...)
```


Configurazione

```
http.httpBasic(  
    Customizer.withDefaults());
```

Esempio REST

```
@Bean
public SecurityFilterChain
filterChain(
    HttpSecurity http)
throws Exception {

    http

        .httpBasic(
            Customizer.withDefaults());

    return http.build();
}
```

Test con curl

```
curl  
  
-u admin:admin  
  
http://localhost:8080/api/studenti
```

Principio di Minimo Privilegio

Ogni utente dovrebbe avere solo i permessi necessari.

Errato

Tutti ADMIN

Corretto

USER
MANAGER
ADMIN

Logging Sicurezza

Importante registrare:

- login
- logout
- tentativi falliti
- modifiche autorizzazioni

Best Practice

Utilizzare BCrypt

Non salvare password in chiaro

Utilizzare HTTPS

Sempre.

Applicare ruoli specifici

Limitare le URL pubbliche

Utilizzare UserDetailsService

Per autenticazione database.

Errori Comuni

Password in chiaro

password

Tutto accessibile

```
.anyRequest()  
.permitAll()
```

Disabilitare la sicurezza

In produzione.

Utilizzare utenti hardcoded

Oltre la fase di sviluppo.

Ruoli troppo permissivi

Esercizi

Esercizio 1

Aggiungere:

```
spring-boot-starter-security
```

ad un progetto.

Esercizio 2

Creare un utente:

admin

con ruolo:

ADMIN

Esercizio 3

Proteggere:

```
/admin/**
```

con:

```
hasRole("ADMIN")
```


Esercizio 4

Configurare:

```
BCryptPasswordEncoder
```

Esercizio 5

Creare un `UserDetailsService` .

Esercizio 6

Configurare Basic Authentication.

Esercizio 7

Proteggere una API REST con ruoli differenti.

Domande di ripasso

1. Qual è la differenza tra autenticazione e autorizzazione?
2. Cos'è Spring Security?
3. Cos'è la Security Filter Chain?
4. A cosa serve `UserDetailsService` ?
5. Perché utilizzare BCrypt?
6. Qual è il ruolo di `PasswordEncoder` ?
7. Cos'è il CSRF?
8. Quando si utilizza Basic Authentication?
9. Perché non bisogna salvare password in chiaro?
10. Cos'è il principio del minimo privilegio?

JWT - JSON Web Token

Introduzione

Nelle moderne applicazioni web e mobile si utilizzano sempre meno le sessioni tradizionali e sempre più sistemi di autenticazione **stateless**.

Esempi:

- API REST
- Angular
- React
- Vue
- Flutter
- Microservizi
- Applicazioni Mobile

La soluzione più diffusa è:

Problema delle Sessioni

Approccio tradizionale:

```
Client
|
Login
|
Server
|
Sessione
```


Il server deve mantenere informazioni sugli utenti connessi.

Svantaggi

- consumo memoria
- scalabilità ridotta
- complessità nei cluster
- difficoltà nei microservizi

Approccio JWT

Con JWT il server non memorizza sessioni.

```
Client
  |
Login
  |
JWT
  |
Client
```

Ogni richiesta contiene il token.

Stateless Authentication

Ogni richiesta è indipendente.

```
Request  
  |  
JWT  
  |  
Autenticazione
```

Il server non deve ricordare nulla.

Cos'è un JWT

Un JWT è una stringa composta da tre parti.

HEADER

.

PAYLOAD

.

SIGNATURE

Esempio

```
eyJhbGciOiJIUzI1NiJ9
.
eyJzdWIiOiJtYXJpbyJ9
.
AbCdEfGhIjKlMnOp
```

Struttura

Header

.

Payload

.

Signature

Header

Contiene informazioni sul token.

Esempio

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Significato

```
alg = algoritmo  
typ = tipo token
```

Payload

Contiene i dati.

Esempio

```
{  
  "sub": "mario",  
  "role": "ADMIN"  
}
```

Claims

Le informazioni contenute nel payload si chiamano:

Claims

Claims Standard

Claim	Significato
sub	Subject
iss	Issuer
exp	Expiration
iat	Issued At
aud	Audience

Subject

```
{  
  "sub": "mario"  
}
```

Utente autenticato.

Expiration

```
{  
  "exp":17300000000  
}
```

Data di scadenza.

Signature

Serve a verificare che il token non sia stato alterato.

Formula

```
Header  
+  
Payload  
+  
Secret Key  
=  
Signature
```

Importante

Il payload NON è cifrato.

Può essere letto.

Non inserire mai:

```
password
```

oppure:

```
dati sensibili
```

Flusso JWT

Login

Username
Password

Verifica

Spring Security

controlla le credenziali.

Token

JWT

viene generato.

Risposta

```
{  
  "token": "eyJ..."  
}
```

Richieste Successive

Header HTTP:

```
Authorization:  
Bearer eyJ...
```

Bearer Token

Formato standard:

```
Authorization:  
Bearer TOKEN
```

Maven

Libreria molto utilizzata:

```
<dependency>

  <groupId>
    io.jsonwebtoken
  </groupId>

  <artifactId>
    jjwt-api
  </artifactId>

  <version>0.12.7</version>
  - - -
</dependency>
```

Secret Key

Chiave utilizzata per firmare il token.

```
jwt.secret=  
myVerySecretKey
```

Generazione Token

Classe utility.

```
@Component  
public class JwtService {  
  
}
```

Creazione

```
public String generateToken(  
    String username)  
{  
}
```

Esempio

```
String token =  
  
    Jwts.builder()  
        .subject(username)  
        .issuedAt(new Date())  
        .expiration(  
            new Date(  
                System.currentTimeMillis()  
                + 3600000))  
        .signWith(secretKey)  
        ---  
        .compact();
```


Risultato

eyJhbGc...

Estrazione Username

```
public String  
extractUsername(  
    String token)  
{  
}
```

Esempio

```
return Jwts.parser()  
    .verifyWith(secretKey)  
    .build()  
    .parseSignedClaims(token)  
    .getPayload()  
    .getSubject();
```

Validazione Token

Verifica:

- firma
- scadenza
- integrità

Metodo

```
public boolean isValid(  
    String token)  
{  
}
```

Login Endpoint

```
@PostMapping("/login")
```

Request

```
{  
  "username": "mario",  
  "password": "password"  
}
```

Risposta

```
{  
  "token": "eyJ..."  
}
```


DTO Login

```
public class LoginRequest {  
    private String username;  
    private String password;  
}
```

DTO Risposta

```
public class LoginResponse {  
    private String token;  
}
```

JwtAuthenticationFilter

Filtro che intercetta le richieste.

```
Request
|
JWT Filter
|
Controller
```

Lettura Header

```
String authHeader =  
    request.getHeader(  
        "Authorization");
```

Verifica Bearer

```
if(authHeader == null ||  
    !authHeader.startsWith(  
        "Bearer "))  
{  
}
```

Estrazione Token

```
String token =  
    authHeader.substring(7);
```

Caricamento Utente

```
String username =  
    jwtService  
        .extractUsername(  
            token);
```

Security Context

Se il token è valido:

```
SecurityContextHolder
```

viene popolato.

Security Configuration

Configurazione tipica.

```
@Bean
public SecurityFilterChain
filterChain(
    HttpSecurity http)
throws Exception
{
}
```

Disabilitare Sessioni

```
.sessionManagement(  
    sm -> sm.sessionCreationPolicy(  
        SessionCreationPolicy.STATELESS))
```

Disabilitare CSRF

Per API REST.

```
.csrf(csrf -> csrf.disable())
```

Endpoint Pubblici

```
.requestMatchers(  
    "/login")  
.permitAll()
```

Endpoint Protetti

```
.anyRequest()  
.authenticated()
```

Inserimento Filtro

```
.addFilterBefore(  
    jwtFilter,  
    UsernamePasswordAuthenticationFilter.class)
```

Refresh Token

Problema:

Token scaduto

Soluzione

Utilizzare:

Access Token

breve durata.

e:

Refresh Token

durata maggiore.

Esempio

```
Access Token  
15 minuti
```

Refresh Token
7 giorni

JWT e Ruoli

Nel token è possibile inserire:

```
{  
  "sub": "mario",  
  "role": "ADMIN"  
}
```

Lettura Ruolo

```
claims.get("role");
```

Architettura Completa

```
Client
|
Login
|
JWT
|
REST API
```

Spring Security + JWT

```
Client
|
JWT Filter
|
Security Context
|
Controller
```

JWT nei Microservizi

Molto utilizzato.

```
Gateway
  |
JWT
  |
Microservizio A

Microservizio B
```

Vantaggi

Stateless

Scalabile

Nessuna Sessione

Perfetto per REST

Compatibile con SPA

Svantaggi

Revoca complessa

Token rubato

Può essere utilizzato fino alla scadenza.

Payload leggibile

Non contiene dati cifrati.

Best Practice

Utilizzare HTTPS

Sempre.

Scadenze brevi

Utilizzare Refresh Token

Firmare con chiavi sicure

Non inserire password

Nel payload.

Salvare token lato client in modo sicuro

Errori Comuni

Token senza scadenza

Secret Key debole

123456

Password nel Payload

Nessuna verifica della firma

Utilizzare HTTP

Anziché HTTPS.

Esercizi

Esercizio 1

Creare un `JwtService` .

Esercizio 2

Generare un token contenente:

```
{  
  "sub": "mario"  
}
```

Esercizio 3

Estrarre lo username da un JWT.

Esercizio 4

Creare endpoint:

```
/login
```

che restituisce il token.

Esercizio 5

Configurare:

```
SessionCreationPolicy.STATELESS
```

Esercizio 6

Realizzare un `JwtAuthenticationFilter` .

Esercizio 7

Proteggere una API REST tramite JWT.

Domande di ripasso

1. Cos'è un JWT?
2. Quali sono le tre parti di un token JWT?
3. Cos'è un Claim?
4. A cosa serve la Signature?
5. Perché JWT è stateless?
6. Qual è il formato dell'header Authorization?
7. A cosa serve `SessionCreationPolicy.STATELESS` ?
8. Cos'è un Refresh Token?
9. Perché non bisogna inserire password nel payload?
10. Quali vantaggi offre JWT rispetto alle sessioni?

OpenAPI e Swagger

Introduzione

Quando un'applicazione REST cresce, diventa fondamentale documentare correttamente le API.

Senza documentazione:

- il frontend non sa come utilizzare gli endpoint
- i client esterni non conoscono parametri e formati
- aumenta il numero di errori di integrazione
- la manutenzione diventa complessa

Per risolvere questo problema si utilizza lo standard:

OpenAPI

e lo strumento più diffuso:

Swagger UI

Cos'è OpenAPI

OpenAPI è uno standard che descrive una API REST.

Definisce:

- endpoint
- parametri
- request
- response
- status code
- autenticazione

Obiettivo

Consentire a persone e strumenti di comprendere automaticamente una API.

Cos'è Swagger

Swagger è un insieme di strumenti che implementano OpenAPI.

I più utilizzati sono:

Swagger UI

Swagger Editor

Swagger Codegen

Swagger UI

Genera automaticamente una documentazione web interattiva.

Esempio:

```
GET /studenti  
POST /studenti  
DELETE /studenti/{id}
```

con possibilità di eseguire chiamate direttamente dal browser.

OpenAPI nel Mondo Spring

La libreria più utilizzata è:

springdoc-openapi

Dipendenza Maven

Spring Boot 3:

```
<dependency>

    <groupId>
        org.springdoc
    </groupId>

    <artifactId>
        springdoc-openapi-starter-webmvc-ui
    </artifactId>

    <version>2.8.9</version>
    ---
</dependency>
```

Avvio

Dopo aver avviato l'applicazione:

```
mvn spring-boot:run
```

URL Swagger

```
http://localhost:8080/swagger-ui.html
```

oppure:

```
http://localhost:8080/swagger-ui/index.html
```

Specifica OpenAPI

Disponibile automaticamente.

```
/v3/api-docs
```

Output

```
{  
  "openapi": "3.0.1"  
}
```

Primo Controller

```
@RestController  
  
@RequestMapping(  
    "/studenti")  
public class StudenteController {  
  
}
```

Endpoint

```
@GetMapping  
public List<StudenteDTO>  
findAll() {  
    return service.findAll();  
}
```


Swagger genera automaticamente la documentazione.

Annotazione @Operation

Permette di descrivere un endpoint.

Esempio

```
@Operation(  
    summary =  
    "Elenco studenti")
```

Metodo

```
@Operation(  
    summary =  
    "Elenco studenti")  
  
@GetMapping  
public List<StudenteDTO>  
findAll() {  
  
}
```

Description

```
@Operation(  
    summary =  
    "Ricerca studente",  
  
    description =  
    "Restituisce lo studente dato il suo ID"  
)
```

Parametri

PathVariable

```
@GetMapping("/{id}")
```

```
public StudenteDTO trova(  
    @PathVariable  
    Long id)  
  
{  
}
```


Swagger documenta automaticamente il parametro.

@Parameter

Personalizzazione.

```
@Parameter(  
    description =  
    "Identificativo studente")
```

Esempio

```
public StudenteDTO trova(  
  
    @Parameter(  
        description =  
        "ID studente")  
  
    @PathVariable  
    Long id)  
  
{  
}
```

DTO

```
public class StudenteDTO {  
    private Long id;  
    private String nome;  
}
```

Swagger genera automaticamente lo schema.

@Schema

Permette di descrivere i campi.

Esempio

```
@Schema(  
    description =  
    "Nome dello studente",  
    example =  
    "Mario")  
private String nome;
```

DTO Completo

```
public class StudenteDTO {  
  
    @Schema(  
        example = "1")  
    private Long id;  
  
    @Schema(  
        example = "Mario")  
    private String nome;  
  
}
```


Documentare le Risposte

ApiResponse

```
@ApiResponse(  
    responseCode = "200",  
    description = "Operazione riuscita")
```

Esempio

```
@Operation(  
    summary =  
    "Ricerca studente")  
  
@ApiResponse(  
    responseCode="200",  
    description="Trovato")  
  
@ApiResponse(  
    responseCode="404",  
    description="Non trovato")
```

ResponseEntity

```
@GetMapping("/{id}")  
  
public ResponseEntity<  
    StudenteDTO>  
trova(  
    @PathVariable Long id)  
{  
}
```

Swagger documenta il tipo restituito.

Raggruppare API

Tag

```
@Tag(  
    name = "Studenti",  
    description =  
        "Gestione studenti")
```

Controller

```
@RestController  
  
@Tag(  
    name = "Studenti")  
  
public class  
StudenteController {  
  
}
```

Risultato

Swagger organizza gli endpoint in sezioni.

Configurazione Generale

Bean OpenAPI

```
@Bean
public OpenAPI openAPI() {
    return new OpenAPI();
}
```

Informazioni API

```
@Bean
public OpenAPI apiInfo() {

    return new OpenAPI()

        .info(
            new Info()

                .title(
                    "Gestione Studenti")

                .version(
                    "1.0")

            ---

            .description(
                "API REST"));
}
```

Output

```
Gestione Studenti  
Versione 1.0
```

Sicurezza

Documentare autenticazione.

JWT

```
@SecurityScheme(  
    name="BearerAuth",  
  
    type=  
        SecuritySchemeType.HTTP,  
  
    scheme="bearer",  
  
    bearerFormat="JWT")
```

Configurazione

```
@Configuration

@SecurityScheme(
    name="BearerAuth",
    type=
        SecuritySchemeType.HTTP,
    scheme="bearer",
    bearerFormat="JWT")
public class
OpenApiConfig {
    ---

}
```

Endpoint Protetto

```
@SecurityRequirement(  
    name="BearerAuth")
```


Esempio

```
@Operation(  
    summary =  
    "Endpoint protetto")  
  
@SecurityRequirement(  
    name="BearerAuth")
```

Pulsante Authorize

Swagger genera automaticamente:

Authorize

per inserire il JWT.

Upload File

Swagger documenta automaticamente.

Endpoint

```
@PostMapping(  
    "/upload")
```

```
public void upload(  
    @RequestParam  
    MultipartFile file)  
  
{  
}
```

OpenAPI YAML

Formato standard.

Esempio

```
openapi: 3.0.0

info:
  title: API Studenti
  version: 1.0
```

OpenAPI JSON

```
{  
  "openapi": "3.0.0"  
}
```


Generazione Client

Da OpenAPI è possibile generare:

- Java Client
- Angular Client
- TypeScript Client
- C# Client
- Python Client

Benefici

Frontend e backend condividono un contratto comune.

Testing API

Swagger permette di eseguire chiamate.

Esempio

Try it out

Execute

Senza Postman.

Architettura

```
Controller
  |
OpenAPI
  |
Swagger UI
```

Best Practice

Documentare tutti gli endpoint

Descrivere DTO

Con:

@Schema

Documentare status code

Con:

```
@ApiResponse
```

Utilizzare Tag

Per organizzare la documentazione.

Documentare la sicurezza

JWT e OAuth2.

Mantenere la documentazione aggiornata

Errori Comuni

Nessuna descrizione

`@GetMapping`

senza documentazione.

DTO privi di Schema

Status code mancanti

API non raggruppate

Sicurezza non documentata

Esercizi

Esercizio 1

Installare:

```
springdoc-openapi
```

Esercizio 2

Documentare un endpoint con:

```
@Operation
```

Esercizio 3

Documentare un DTO tramite:

```
@Schema
```


Esercizio 4

Documentare:

404 Not Found

con:

@ApiResponse

Esercizio 5

Configurare informazioni globali API.

Esercizio 6

Configurare JWT tramite:

```
@SecurityScheme
```

Esercizio 7

Documentare completamente una API CRUD Studenti.

Domande di ripasso

1. Cos'è OpenAPI?
2. Cos'è Swagger UI?
3. Quale URL mostra la documentazione?
4. A cosa serve `@Operation` ?
5. A cosa serve `@Schema` ?
6. A cosa serve `@ApiResponse` ?
7. Come si documenta JWT?
8. Cos'è `/v3/api-docs` ?
9. Quali vantaggi offre la generazione automatica dei client?
10. Perché è importante mantenere aggiornata la documentazione?

Docker per Java

Introduzione

Negli ultimi anni il deployment delle applicazioni è cambiato radicalmente.

In passato era necessario:

- installare Java
- configurare il server
- configurare il database
- installare librerie
- gestire conflitti tra versioni

Con Docker l'applicazione viene distribuita insieme al proprio ambiente di esecuzione.

Applicazione
+
Dipendenze
+
Configurazione

Cos'è Docker

Docker è una piattaforma che consente di creare ed eseguire applicazioni all'interno di contenitori isolati.

Vantaggi

- portabilità
- semplicità di deployment
- isolamento
- scalabilità
- riproducibilità

Problema Tradizionale

Funziona sul mio PC...

```
Developer
  |
Funziona
  |
Server Produzione
  |
Non funziona
```

Soluzione Docker

```
Stessa immagine
|
Sviluppo
|
Test
|
Produzione
```

Virtual Machine vs Container

Virtual Machine

```
graph TD; Hardware --> Hypervisor; Hypervisor --> Guest_OS[Guest OS]; Guest_OS --> Applicazione
```

Hardware
|
Hypervisor
|
Guest OS
|
Applicazione

Docker

```
graph TD; Hardware --> Host_OS[Host OS]; Host_OS --> Docker_Engine[Docker Engine]; Docker_Engine --> Container;
```

Hardware
|
Host OS
|
Docker Engine
|
Container

Vantaggi dei Container

- avvio rapido
- minore consumo memoria
- immagini leggere
- maggiore densità applicativa

Concetti Fondamentali

Image

Un'immagine è un modello immutabile.

```
springboot-app:1.0
```

Container

Un container è un'istanza in esecuzione di un'immagine.

```
graph TD; Image --> Container;
```

Image
|
Container

Registry

Archivio di immagini.

Esempi:

- Docker Hub
- GitHub Container Registry
- AWS ECR
- Azure Container Registry

Docker Hub

Repository pubblico.

```
https://hub.docker.com
```

Installazione

Verifica:

```
docker --version
```

Output:

```
Docker version 28.x
```

Primo Container

```
docker run hello-world
```

Verifica Container

```
docker ps
```


Tutti i Container

```
docker ps -a
```

Immagini Locali

`docker images`

Scaricare una Immagine

```
docker pull openjdk:21
```

Avviare un Container

```
docker run openjdk:21
```

Modalità Interattiva

```
docker run -it openjdk:21 bash
```

Verificare Java

```
java -version
```

Spring Boot e Docker

Applicazione:

```
app.jar
```

Dockerfile

File che descrive come costruire un'immagine.

Esempio

```
FROM eclipse-temurin:21

COPY target/app.jar app.jar

ENTRYPOINT [
    "java",
    "-jar",
    "/app.jar"
]
```

Significato

FROM

Immagine base.

```
FROM eclipse-temurin:21
```

COPY

Copia file.

```
COPY target/app.jar app.jar
```

ENTRYPOINT

Comando eseguito all'avvio.

```
ENTRYPOINT [  
  "java",  
  "-jar",  
  "app.jar"  
]
```

Build Applicazione

```
mvn clean package
```

Output:

```
target/app.jar
```

Creazione Immagine

```
docker build -t scuola-app .
```


Verifica

docker images

Avvio Container

```
docker run scuola-app
```

Esposizione Porte

Spring Boot:

```
server.port=8080
```

Container:

```
docker run -p 8080:8080 scuola-app
```

Significato

HOST:8080

|

CONTAINER:8080

Container in Background

```
docker run -d scuola-app
```

Log Container

```
docker logs CONTAINER_ID
```

Stop Container

```
docker stop CONTAINER_ID
```


Rimuovere Container

```
docker rm CONTAINER_ID
```

Rimuovere Immagine

```
docker rmi scuola-app
```

Variabili Ambiente

Molto utilizzate.

Esempio

```
docker run  
  
-e DB_HOST=localhost  
  
scuola-app
```

Spring Boot

```
spring.datasource.url=  
${DB_URL}
```

Volume

Permette di persistere dati.

Esempio

```
docker run  
-v dati:/app/data  
scuola-app
```

Docker Network

Container che comunicano tra loro.


```
Spring Boot
  |
  Docker Network
  |
  MySQL
```

Creazione

```
docker network create scuola-net
```

Docker Compose

Gestisce più container.

Scenario

```
Spring Boot  
|  
MySQL
```

docker-compose.yml

```
services:  
  app:  
    image: scuola-app  
  db:  
    image: mysql:8
```

Esempio Completo

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_DATABASE: scuola
      MYSQL_ROOT_PASSWORD: root
  app:
    image: scuola-app
    ports:
      - "8080:8080"
    depends_on:
      - db
```

Avvio Compose

```
docker compose up
```

Background

```
docker compose up -d
```


Arresto

```
docker compose down
```

Spring Boot + MySQL

Configurazione:

```
spring.datasource.url=  
jdbc:mysql://db:3306/scuola
```

Importante

```
db
```

è il nome del servizio Compose.

Multi Stage Build

Riduce dimensioni immagine.

Dockerfile

```
FROM maven:3.9-eclipse-temurin-21
AS build

COPY . .

RUN mvn package
```

Runtime

```
FROM eclipse-temurin:21
```

```
COPY --from=build  
/target/app.jar  
app.jar
```

Benefici

- immagini più piccole
- maggiore sicurezza
- build automatica

Health Check

Verifica stato container.

Dockerfile

HEALTHCHECK

CMD curl -f
http://localhost:8080/actuator/health

Spring Boot Actuator

```
/actuator/health
```

Docker e CI/CD

Pipeline tipica:

```
Git
|
Build
|
Test
|
Docker Build
|
Docker Registry
|
Produzione
```

Kubernetes

Docker è spesso utilizzato insieme a:

Kubernetes

Architettura

```
graph TD; A[Docker Image] --> B[Kubernetes]; B --> C[Cluster];
```

Docker Image
|
Kubernetes
|
Cluster

Best Practice

Utilizzare immagini ufficiali

```
eclipse-temurin
```

Evitare root

Utilizzare Multi Stage Build

Configurare Health Check

Utilizzare variabili ambiente

Salvare dati nei volumi

Errori Comuni

Salvare password nel Dockerfile

Errato:

```
ENV PASSWORD=root
```

Immagini enormi

Nessun volume

Perdita dati alla rimozione del container.

Container monolitici

Troppi servizi nello stesso container.

Nessun Health Check

Esercizi

Esercizio 1

Installare Docker e verificare:

```
docker --version
```

Esercizio 2

Eseguire:

```
docker run hello-world
```

Esercizio 3

Creare un Dockerfile per una applicazione Spring Boot.

Esercizio 4

Costruire una immagine:

```
docker build -t scuola-app .
```

Esercizio 5

Pubblicare la porta:

```
-p 8080:8080
```


Esercizio 6

Creare un file:

```
docker-compose.yml
```

contenente:

- Spring Boot
- MySQL

Esercizio 7

Realizzare una build multi-stage.

Domande di ripasso

1. Cos'è Docker?
2. Qual è la differenza tra Image e Container?
3. Cos'è un Dockerfile?
4. A cosa serve `docker build` ?
5. A cosa serve `docker run` ?
6. Cos'è Docker Compose?
7. Perché utilizzare i volumi?
8. Qual è il vantaggio delle build multi-stage?
9. A cosa serve un Health Check?
10. Come comunica Spring Boot con MySQL in Docker Compose?

Testing Spring Boot

Introduzione

In un'applicazione Spring Boot professionale il testing è fondamentale per garantire:

- qualità del codice
- affidabilità
- manutenibilità
- sicurezza delle modifiche
- prevenzione delle regressioni

Nel mondo Spring esistono diversi livelli di test:

```
Unit Test
|
Integration Test
|
Web Layer Test
|
End-To-End Test
```

Ogni livello verifica aspetti differenti dell'applicazione.

Strategia di Testing

La strategia più diffusa è la Test Pyramid.



Unit Test

Verificano una singola classe.

Integration Test

Verificano l'integrazione tra componenti.

End-To-End Test

Verificano l'intera applicazione.

Dipendenze Maven

Spring Boot fornisce uno starter dedicato.

```
<dependency>

  <groupId>
    org.springframework.boot
  </groupId>

  <artifactId>
    spring-boot-starter-test
  </artifactId>

  <scope>test</scope>
  - - -

</dependency>
```

Cosa Include

Lo starter contiene:

- JUnit 5
- Mockito
- Spring Test
- MockMvc
- AssertJ
- JSONAssert

Unit Test Tradizionale

Classe:

```
@Service
public class CalcolatriceService {

    public int somma(
        int a,
        int b)
    {
        return a + b;
    }
    ---
}
```

Test

```
class CalcolatriceServiceTest {  
  
    @Test  
    void shouldAddNumbers() {  
  
        CalcolatriceService service =  
            new CalcolatriceService();  
  
        assertEquals(  
            5,  
            service.somma(2, 3));  
  
    }  
    ---  
}
```

Testing Service con Mockito

Repository

```
public interface  
StudenteRepository {  
  
}
```


Service

```
@Service
public class StudenteService {

    private final
    StudenteRepository repository;

}
```

Test

```
@ExtendWith(  
    MockitoExtension.class)  
  
class StudenteServiceTest {  
  
    @Mock  
    private StudenteRepository repository;  
  
    @InjectMocks  
    private StudenteService service;  
  
}
```

MockBean

Mockito puro funziona bene per Unit Test.

Spring introduce:

```
@MockBean
```

per sostituire Bean reali.

@SpringBootTest

Carica l'intera applicazione Spring.

Esempio

```
@SpringBootTest  
class ApplicationTest {  
  
}
```

Cosa Succede

Spring avvia:

```
ApplicationContext
```

completo.

Include

- Bean
- Service
- Repository
- Configurazioni

Quando Utilizzarlo

Per verificare:

- integrazione tra componenti
- configurazione Spring
- bean wiring

Esempio

```
@SpringBootTest  
  
class StudenteServiceIT {  
  
    @Autowired  
    private StudenteService service;  
  
}
```

Verifica Bean

```
@Test
void contextLoads() {
    assertNotNull(service);
}
```

Limite

Più lento dei normali Unit Test.

Web Layer Testing

Spesso non serve avviare tutta l'applicazione.

@WebMvcTest

Carica solo il layer MVC.

Controller

```
@RestController  
  
@RequestMapping(  
    "/studenti")  
public class StudenteController {  
  
}
```

Test

```
@WebMvcTest(  
    StudenteController.class)  
  
class StudenteControllerTest {  
  
}
```

MockMvc

Permette di simulare richieste HTTP.

Iniezione

```
@Autowired  
private MockMvc mockMvc;
```

GET

```
@Test
void shouldReturnStudents()
throws Exception {

    mockMvc.perform(
        get("/studenti"))
        .andExpect(
            status().isOk());

}
```

POST

```
mockMvc.perform(  
    post("/studenti")  
        .contentType(  
            MediaType.APPLICATION_JSON)  
        .content(json));
```

Verifica JSON

```
.andExpect(  
    jsonPath("$.nome")  
    .value("Mario"));
```

MockBean nel Controller Test

```
@MockBean  
private StudenteService service;
```

Esempio Completo

```
@WebMvcTest(  
    StudenteController.class)  
  
class StudenteControllerTest {  
  
    @Autowired  
    private MockMvc mockMvc;  
  
    @MockBean  
    private StudenteService service;  
  
}
```

Test REST Completi

Quando si vuole testare davvero il server.

@SpringBootTest

```
@SpringBootTest(  
    webEnvironment =  
    SpringBootTest.WebEnvironment  
        .RANDOM_PORT)
```


TestRestTemplate

Client HTTP integrato.

Iniezione

```
@Autowired  
private TestRestTemplate rest;
```

Richiesta

```
ResponseEntity<String>  
response =  
  
rest.getForEntity(  
    "/studenti",  
    String.class);
```

Verifica

```
assertEquals(  
    HttpStatus.OK,  
    response.getStatusCode());
```

Testing Repository

Configurazione

```
@DataJpaTest
```

Cosa Carica

Solo:

- Entity
- Repository
- Hibernate
- Database embedded

Test

```
@DataJpaTest  
  
class StudentRepositoryTest {  
  
}
```


Entity Manager

Disponibile automaticamente.

```
@Autowired  
private EntityManager em;
```

Esempio

```
Studente s =  
    new Studente();  
  
em.persist(s);
```

Query Test

```
Optional<Studente>  
result =  
  
repository.findById(  
    s.getId());
```

Assert

```
assertTrue(  
    result.isPresent());
```

Testing Sicurezza

Spring Security può essere testato.

Utente Simulato

```
@WithMockUser(  
    username="mario",  
    roles={"USER"})
```

Test

```
@Test  
void shouldAccess()  
throws Exception {  
  
}
```

Verifica Accesso Negato

```
.andExpect(  
    status().isForbidden());
```


Testing JSON

Spring include Jackson.

ObjectMapper

```
@Autowired  
private ObjectMapper mapper;
```

Conversione

```
String json =  
    mapper.writeValueAsString(  
        dto);
```

AssertJ

Libreria di assertion moderna.

Esempio

```
assertThat(lista)  
    .hasSize(2)  
    .isNotEmpty();
```

Testcontainers

Problema:

H2
≠
MySQL

Soluzione

Testcontainers

Obiettivo

Utilizzare database reali nei test.

Dipendenza

```
<dependency>  
  <groupId>  
    org.testcontainers</groupId>  
  
  <artifactId>  
    mysql</artifactId>  
  
  <scope>test</scope>  
</dependency>
```

Container MySQL

```
@Testcontainers
class RepositoryTest {

    @Container

    static MySQLContainer<?>
        mysql =
            new MySQLContainer<>(
                "mysql:8");

}
```

Benefici

Test eseguiti su:

MySQL reale

Slice Testing

Spring Boot offre test specializzati.

Annotazione	Layer
@WebMvcTest	MVC
@DataJpaTest	JPA
@JsonTest	JSON
@SpringBootTest	Tutto

@JsonTest

Verifica serializzazione.

Test

```
@JsonTest  
class StudenteJsonTest {  
  
}
```

Coverage

Misura quanto codice è testato.

Strumenti

JaCoCo

SonarQube

Obiettivo

Indicativamente:

70% - 90%

Best Practice

Multi Unit Test

Pochi Integration Test

Test Veloci

Mockare servizi esterni

Utilizzare Testcontainers

Per database.

Nomi Descrittivi

```
shouldReturnStudentWhenIdExists()
```

Errori Comuni

Solo Test Manuali

Troppi @SpringBootTest

Rallentano la suite.

Nessun Test Repository

Nessun Test Sicurezza

Database Produzione nei Test

Mai.

Esercizi

Esercizio 1

Creare un Unit Test per:

StudenteService

utilizzando Mockito.

Esercizio 2

Creare un:

```
@WebMvcTest
```

per un Controller REST.

Esercizio 3

Utilizzare:

MockMvc

per verificare un endpoint.

Esercizio 4

Creare un:

```
@DataJpaTest
```

per un Repository.

Esercizio 5

Utilizzare:

```
@WithMockUser
```

per testare Spring Security.

Esercizio 6

Configurare:

Testcontainers

con MySQL.

Esercizio 7

Realizzare una suite completa composta da:

```
Unit Test  
MVC Test  
Repository Test  
Integration Test
```

per una API CRUD Studenti.

Domande di ripasso

1. Qual è la differenza tra Unit Test e Integration Test?
2. Quando utilizzare `@SpringBootTest` ?
3. Quando utilizzare `@WebMvcTest` ?
4. A cosa serve `MockMvc` ?
5. Qual è il ruolo di `@DataJpaTest` ?
6. Cos'è `@MockBean` ?
7. A cosa serve `TestRestTemplate` ?
8. Perché utilizzare Testcontainers?
9. Qual è il vantaggio di `@WithMockUser` ?
10. Quali livelli di test dovrebbe avere una applicazione Spring Boot?

Microservizi

Introduzione

Per molti anni la maggior parte delle applicazioni è stata sviluppata come:

Monolite

ovvero un'unica applicazione contenente:

- autenticazione
- utenti
- catalogo
- pagamenti
- report
- amministrazione

Con la crescita del sistema emergono problemi di:

- manutenzione
- scalabilità
- deployment
- affidabilità

Per questo motivo molte aziende hanno adottato l'architettura:

Microservices

Cos'è un Microservizio

Un microservizio è una applicazione autonoma che implementa una singola funzionalità di business.

Esempio:

Servizio Utenti

Servizio Corsi

Servizio Esami

Servizio Notifiche

Ogni servizio è indipendente.

Monolite

```
+-----+
| Applicazione |
+-----+
| Utenti      |
| Corsi       |
| Esami       |
| Pagamenti   |
| Report      |
+-----+
```

Microservizi

```
+-----+
| Utenti |
+-----+

+-----+
| Corsi  |
+-----+

+-----+
| Esami  |
+-----+
- - -

+-----+
| Report |
+-----+
```

Obiettivo

Separare i domini applicativi.

Principio:

Un servizio
una responsabilità

Domain Driven Design

I microservizi spesso derivano dai concetti di:

DDD
Domain Driven Design

Esempio

Sistema scolastico:

Studenti

Docenti

Esami

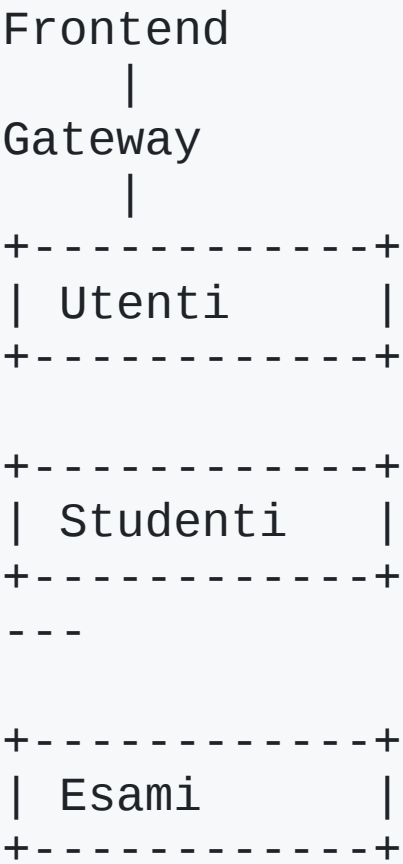
Pagamenti

Ogni dominio può diventare un microservizio.

Architettura Tipica

```
Frontend
  |
API Gateway
  |
+-----+
| Servizi |
+-----+
```

Esempio Completo



Caratteristiche

Ogni microservizio:

- ha il proprio database
- ha il proprio deploy
- può essere aggiornato indipendentemente
- può scalare autonomamente

Database per Servizio

Principio fondamentale.

Corretto

```
Utenti  
  |  
Database A
```

```
Studenti  
  |  
Database B
```

Errato

```
Tutti i servizi  
|  
Database unico
```

Vantaggi

Scalabilità

Possiamo scalare solo il servizio necessario.

Esempio

Utenti
1 istanza

Catalogo
10 istanze

Deploy Indipendente

Aggiornare il servizio:

Notifiche

senza fermare:

Utenti

Tecnologie Diverse

Ogni team può utilizzare tecnologie differenti.

Esempio

Utenti -> Java

Analytics -> Python

Frontend -> Angular

Fault Isolation

Errore in:

Notifiche

non blocca necessariamente:

Pagamenti

Svantaggi

I microservizi introducono complessità.

Comunicazione Remota

```
Servizio A  
  |  
REST  
  |  
Servizio B
```


Gestione Rete

Possibili problemi:

- timeout
- latenza
- servizi non disponibili

Debug Distribuito

Più difficile.

Monitoraggio

Occorre monitorare molti processi.

Quando NON Usare i Microservizi

Per piccoli progetti.

Meglio Monolite

Applicazione semplice

Team piccolo

Poche funzionalità

Comunicazione tra Servizi

REST

Approccio più diffuso.

HTTP
JSON

Esempio

```
GET  
/api/studenti/10
```


Client REST

Spring offre:

RestTemplate

(storico)

WebClient

(moderno)

WebClient

```
WebClient client =  
    WebClient.create();
```

Chiamata

```
String response =  
client.get()  
    .uri(  
        "http://servizio")  
    .retrieve()  
    .bodyToMono(  
        String.class)  
    .block();
```

Comunicazione Asincrona

Alternativa al REST.

Message Broker

RabbitMQ

Kafka

ActiveMQ

Flusso

```
Servizio A
  |
Evento
  |
Kafka
  |
Servizio B
```

Event Driven Architecture

I servizi comunicano tramite eventi.

Evento

StudenteCreato

Consumatore

Servizio Notifiche

API Gateway

Punto di ingresso unico.

Senza Gateway



Con Gateway

```
Frontend
|
Gateway
|
Servizi
```

Vantaggi Gateway

- autenticazione
- logging
- routing
- rate limiting
- monitoraggio

Spring Cloud Gateway

Soluzione moderna.

Dipendenza

```
<dependency>
  <groupId>
    org.springframework.cloud
  </groupId>

  <artifactId>
    spring-cloud-starter-gateway
  </artifactId>

</dependency>
```

Routing

```
routes:
```

```
- id: studenti
```

```
  uri:
```

```
    http://localhost:8081
```


Service Discovery

Problema:

Dove si trova il servizio?

Soluzione

Registro centrale.

Eureka Server

Architettura

```
Eureka
|
Servizio A

Servizio B

Servizio C
```

Registrazione

Ogni servizio comunica:

Sono disponibile

al registry.

Config Server

Gestione centralizzata delle configurazioni.

Problema

```
50 servizi  
50 file application.yml
```

Soluzione

Spring Cloud Config

Circuit Breaker

Problema:

Servizio B offline

Effetto

Servizio A bloccato

Soluzione

Circuit Breaker

Resilience4J

Libreria più utilizzata.

Dipendenza

```
<dependency>
  <groupId>
    io.github.resilience4j
  </groupId>
  <artifactId>
    resilience4j-spring-boot3
  </artifactId>
</dependency>
```

Utilizzo

```
@CircuitBreaker(  
    name = "studentService")
```

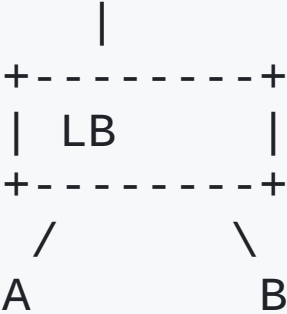
Fallback

```
public String fallback(  
    Exception ex)  
{  
    return "Servizio non disponibile";  
}
```

Load Balancing

Distribuzione del carico.

Client



Osservabilità

In un sistema distribuito è fondamentale.

Logging Centralizzato

Strumenti:

Elastic Stack

Grafana

Loki

Distributed Tracing

Permette di seguire una richiesta.

Strumenti

Zipkin

OpenTelemetry

Jaeger

Containerizzazione

I microservizi sono quasi sempre eseguiti tramite:

Docker

Orchestrazione

In produzione si utilizza spesso:

Kubernetes

Spring Cloud

Ecosistema dedicato ai microservizi.

Componenti

Gateway

Eureka

Config Server

OpenFeign

Resilience4J

OpenFeign

Client REST dichiarativo.

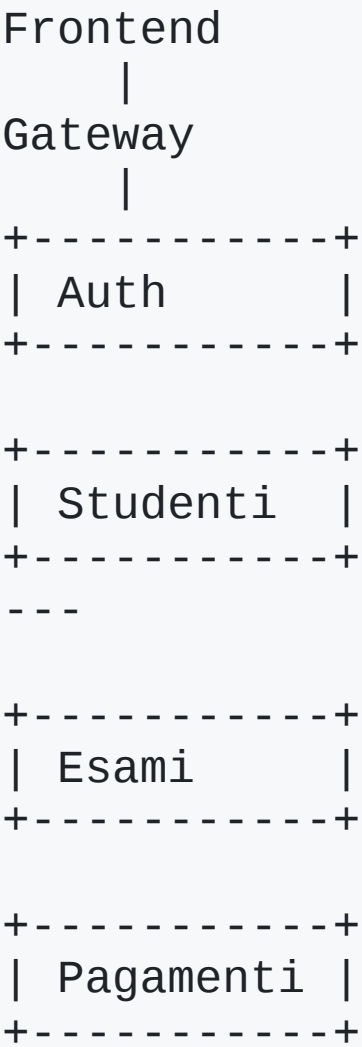
Esempio

```
@FeignClient(  
    name = "studenti")  
public interface  
StudenteClient {  
  
}
```

Metodo

```
@GetMapping("/{id}")  
StudenteDTO trova(  
    Long id);
```

Architettura Enterprise



Monolite o Microservizi?

Monolite	Microservizi
Semplice	Complesso
Deploy unico	Deploy multipli
Facile debug	Debug distribuito
Ideale piccoli progetti	Ideale grandi sistemi

Best Practice

Partire da un monolite ben progettato

Estrarre microservizi solo quando serve

Database separato per servizio

Utilizzare Gateway

Applicare Circuit Breaker

Centralizzare logging e monitoraggio

Errori Comuni

Microservizi troppo piccoli

Database condiviso

Comunicazione sincrona eccessiva

Nessun monitoraggio

Nessuna resilienza

Esercizi

Esercizio 1

Progettare una piattaforma scolastica composta da:

Studenti

Docenti

Esami

come microservizi separati.

Esercizio 2

Creare una comunicazione REST tra due servizi Spring Boot.

Esercizio 3

Utilizzare:

```
WebClient
```

per consumare una API remota.

Esercizio 4

Configurare un semplice API Gateway.

Esercizio 5

Creare un client:

OpenFeign

Esercizio 6

Applicare:

@CircuitBreaker

con Resilience4J.

Esercizio 7

Disegnare una architettura completa con:

Gateway
Auth
Studenti
Esami
Notifiche

Domande di ripasso

1. Cos'è un microservizio?
2. Quali vantaggi offre rispetto a un monolite?
3. Perché ogni microservizio dovrebbe avere il proprio database?
4. Qual è il ruolo di un API Gateway?
5. Cos'è Service Discovery?
6. A cosa serve Eureka?
7. Cos'è un Circuit Breaker?
8. Qual è il ruolo di Resilience4J?
9. Cos'è OpenFeign?
10. Quando conviene scegliere un monolite invece dei microservizi?

Fine del Livello 04

Con questo capitolo si conclude il percorso:

Livello 04
Java Enterprise

Argomenti coperti:

- JDBC
- Maven
- Modularità
- JUnit
- Mockito
- Logging
- Architetture Applicative
- Spring Core